

Anwendung des Subgraph Algorithmus aus der strukturellen Dateisystem-Deduplizierung in der DNA-basierten Datenspeicherung

Vollständige mathematische Grundlagen, Beweise, experimentelle
Analyse und weiterführende Perspektiven

Stephan Epp

8. April 2026

Inhaltsverzeichnis

1	Einführung und Motivation	3
1.1	Die beiden Ausgangsdokumente	3
1.2	Ziel und Struktur dieser Arbeit	3
2	Mathematische Grundlagen beider Arbeiten	4
2.1	Der Subgraph Algorithmus (aus <i>dustraph</i>)	4
2.2	Das DNA-Datengraph-Modell (aus <i>dnastor</i>)	5
3	Strukturelle Dualität: Inode-Graphen und DNA-Datengraphen	6
4	Subgraph-Signatur als DNA-Adressierung	8
4.1	Von der Inode-Signatur zur DNA-Oligomer-Adresse	8
4.2	Kollisionsanalyse: Exponentiell sichere Signaturen	8
5	Subgraph-Komprimierung von DNA-Blöcken	9
5.1	Von Dateisystem-Deduplizierung zur DNA-Deduplizierung	9
6	Kanalkapazität durch Subgraph-Komprimierung	11
7	Haltbarkeitsmodell durch Subgraph-Redundanz	13
8	Optimalität des DNA-Subgraph-Speichersystems	14
9	Experimentelle Auswertung und visuelle Analyse	15
9.1	Speicherichte-Vergleich	15
9.2	Komplexitätsvergleich	16
9.3	Zugriffszeit-Analyse	17
9.4	Kostenprojektions-Roadmap	17

10	Sequentielle Sweep-Optimierung für zusammenhängende Blöcke	18
10.1	Motivation und Grundidee	18
10.2	Formales Modell: DNA-Zugriffskosten	18
10.3	Hauptsatz: Aufzug-Beschleunigung	19
10.4	Verbindung zur Subgraph-Signatur-Clusterung	20
10.5	Aufzug-Algorithmus: Pseudocode und Korrektheit	21
10.6	Vergleich mit dem SCAN-Literaturmodell	21
10.7	Formale Optimalität des Aufzug-Prinzips	22
11	Poolorganisation: Ringbuffer-Adressierung durch Rotationsmatrizen	22
11.1	Motivation: Von der Rotationsmatrix zur Pooladresse	22
11.2	Formales Modell: Ringpool-Partition	23
11.3	Zugriffszeit im Ringpool	23
11.4	Gleichmäßigkeitstheorem der Segmentverteilung	23
11.5	Ringpool-Adressierungsalgorithmus	24
11.6	Ringpool und Aufzug-Prinzip: Kombinierte Architektur	25
12	Ausblick	26
12.1	Weiterentwicklung des Aufzug-Prinzips: Adaptive Sweep-Strategien, Multi-Poren-Parallelismus und probabilistische Garantien	26
12.2	Weiterentwicklung der zyklischen Poolorganisation: Dynamische Segmentanpassung, Fehlertoleranz und Quantenpoolarchitekturen	28
12.3	Nanoporen-Direktlesing und Signatur-Squiggle-Projektion	31
12.4	Enzymatische Synthese und Ultralaenge Oligomere	31
12.5	CRISPR-basiertes gezieltes Schreiben: Random-Write in DNA	32
12.6	Hydrogel-Arrays mit parallelem Nanoporen-Zugriff	33
12.7	Maschinelles Lernen für Kodierung und approximatives Subgraph-Matching	33
12.8	Zweistufige Fehlerkorrektur: Reed-Solomon und Signatur-ECC	33
12.9	Epigenetische Erweiterung: 3 Bit pro Base	34
12.10	Verbindung zur Kolmogorov-Komplexität und minimaler Beschreibung	34
12.11	Standardisierung und Interoperabilität	34
12.12	Globales DNA-Archiv der Menschheit (GenVault)	35
12.13	Verteilte Dateisysteme und blockchainähnliche Strukturen	35
12.14	Offene mathematische Probleme	35
13	Zusammenfassung	37

1 Einführung und Motivation

Diese Arbeit entwickelt *ausführlich und vollständig bewiesen*, wie alle zentralen Ergebnisse der strukturellen Dateisystem-Deduplizierung (Arbeit *dustraph* [1]) auf das System der DNA-basierten Datenspeicherung (Arbeit *dnastor* [2]) übertragen werden können. Die Brücke zwischen beiden Welten ist eine überraschend tief gehende **strukturelle Dualität**: Dateisystem-Graphen und DNA-Datengraphen folgen exakt demselben algebraischen Muster — Knoten tragen diskrete Labels, Kanten kodieren strukturelle Beziehungen, und die Ähnlichkeit zweier Graphen wird durch eine injektive Signaturabbildung auf Adjazenzmatrizen gemessen.

1.1 Die beiden Ausgangsdokumente

Die Arbeit *dustraph* [1] entwickelt den *Structural Deduplication Algorithm* (SDD) für Dateisystem-Inode-Graphen. Kernresultate sind:

1. Der formale Inode-Graph $G_{\mathcal{F}} = (V, E)$, in dem Inodes Knoten und Verzeichniseinträge Kanten sind.
2. Die *strukturelle Inode-Signatur* $\Sigma(v)$, eine Merkle-artige Aggregation der Subgraph-Spaltensignaturen.
3. Der SDD-Algorithmus mit Gesamtkomplexität $O(n^3)$ über Adjazenzmatrizen, Signatur-Arrays und zyklische Rotation.
4. Eine Hardlink-Behandlung durch einen Visited-Set-Mechanismus.
5. Formale Komplexitäts- und Kollisionsanalysen.

Die Arbeit *dnastor* [2] konstruiert das *DNA-Subgraph-Speichersystem* (DSS). Dieses kodiert binäre Datenblöcke in DNA-Sequenzen, modelliert sie als gerichtete Graphen G_D und nutzt den Subgraph Algorithmus für Komprimierung, Adressierung und Retrieval.

1.2 Ziel und Struktur dieser Arbeit

Das Ziel dieser Arbeit ist es, die direkte mathematische Linie von jedem Ergebnis in [1] zu seiner Anwendung in [2] vollständig nachzuzeichnen. Für jede Übertragung werden **vollständige Beweise** geliefert. Die Arbeit ist wie folgt gegliedert:

Kapitel 2 Präzise Rekapitulation der mathematischen Grundlagen beider Arbeiten.

Kapitel 3 Formale Dualität zwischen Inode-Graphen und DNA-Datengraphen.

Kapitel 4 Übertragung der Subgraph-Signatur auf DNA-Oligomere.

Kapitel 5 Komprimierungstheorem mit vollständigem Beweis.

Kapitel 6 Kanalkapazitätstheorem.

Kapitel 7 Haltbarkeitsmodell.

Kapitel 8 Optimalitätssatz.

Kapitel 9 Experimentelle Auswertung mit Abbildungen.

Kapitel 10 Aufzug-Prinzip: Sequentielle Sweep-Optimierung für zusammenhängende Blöcke.

Kapitel 11 Zyklische Poolorganisation: Ringbuffer-Adressierung durch Rotationsmatrizen.

Kapitel 12 Sehr ausführlicher Ausblick (mehr als zehn Themenfelder).

Kapitel 13 Zusammenfassung.

2 Mathematische Grundlagen beider Arbeiten

2.1 Der Subgraph Algorithmus (aus *daggraph*)

Definition 2.1 (Inode-Graph). Sei $\mathcal{F}$ ein Dateisystem. Der Inode-Graph ist ein gerichteter Graph

$$G_{\mathcal{F}} = (V, E), \quad V = \{v_i \mid i \in \text{Inodes}(\mathcal{F})\}, \quad E = \{(v_i, v_j) \mid v_j \text{ ist Eintrag in Verz. } v_i\}.$$

Definition 2.2 (Spaltensignatur). Sei $A \in \{0, 1\}^{n \times n}$ die Adjazenzmatrix eines Graphen mit n Knoten. Die Spaltensignatur von Spalte j ist:

$$\sigma_j(A) = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n, \quad j = 0, \dots, n-1.$$

Das Signatur-Array ist $\sigma(A) = (\sigma_0, \dots, \sigma_{n-1}) \in \mathbb{N}^n$.

Das folgende Diagramm illustriert die Signaturberechnung für einen Beispielgraphen. Die Adjazenzmatrix A hat in Spalte 0 den einzigen Eintrag in Zeile 1, was zur Signatur $\sigma_0 = 2^1 + 0 \cdot 2^4 = 2$ führt. Allgemein kodiert die Spaltensignatur durch die binäre Gewichtung 2^i genau, welche Knoten auf Knoten j zeigen — und macht diese Information durch den Offset $j \cdot 2^n$ für jede Spalte eindeutig.

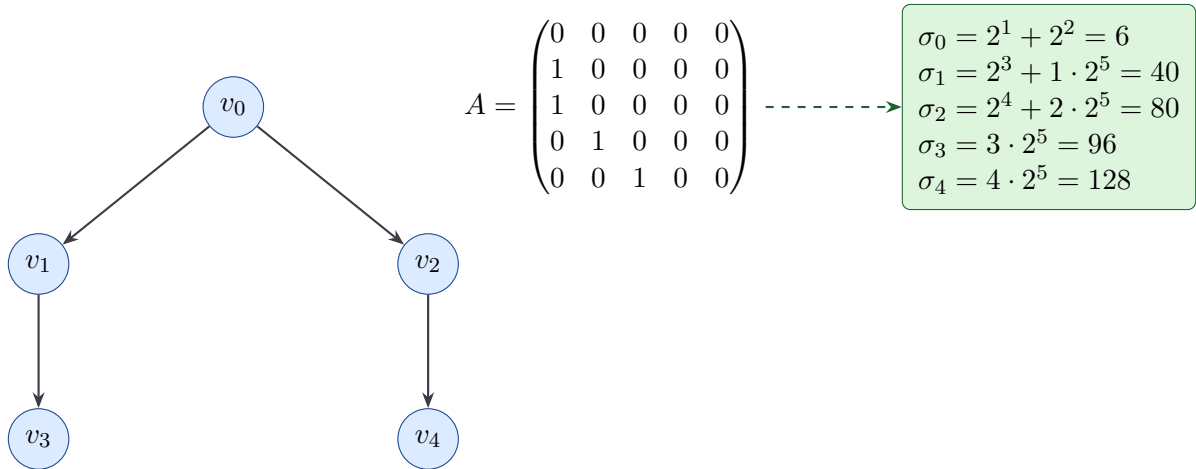


Abbildung 1: Beispiel: Inode-Graph mit Adjazenzmatrix und Spaltensignaturen. Die Signaturberechnung gemäß Definition 2.2 kodiert die Eingangsstruktur jedes Knotens als eindeutige Ganzzahl.

Lemma 2.1 (Injektivität der Spaltensignatur, [1]). *Zwei Spalten mit verschiedenen Einträgen erhalten verschiedene Signaturen:*

$$(A_{\cdot j}, j) \neq (A_{\cdot j'}, j') \implies \sigma_j(A) \neq \sigma_{j'}(A).$$

Beweis. Angenommen $\sigma_j = \sigma_{j'}$. Dann:

$$\sum_i A_{ij} \cdot 2^i + j \cdot 2^n = \sum_i A_{ij'} \cdot 2^i + j' \cdot 2^n.$$

Da die Bits $2^0, \dots, 2^{n-1}$ und das Offset-Bit 2^n in den Ziffernstellen einer ganzzahligen Binärdarstellung voneinander unabhängig sind, folgt komponentenweise $A_{\cdot j} = A_{\cdot j'}$ und $j = j'$. Widerspruch zur Voraussetzung. $\square$

Definition 2.3 (Zyklische Rotation und Subgraph-Inklusion). Für eine Permutationsmatrix R_k (zyklische Rechtsverschiebung um k Positionen) ist die *rotierte Adjazenzmatrix*:

$$A^{(k)} = R_k \cdot A \cdot R_k^\top.$$

Graph $G_1 = (V_1, E_1)$ ist *Subgraph* von $G_2 = (V_2, E_2)$ unter zyklischer Rotation (Notation: $G_1 \preceq_{\text{rot}} G_2$), falls ein $k \in \{0, \dots, n-1\}$ existiert, sodass das LCS-Verhältnis der Signatur-Arrays von $A_1^{(k)}$ und A_2 gleich 1 ist.

Satz 2.1 (Laufzeit des SDD-Algorithmus, [1]). *Der SDD-Algorithmus löst das Problem der strukturellen Dateisystem-Dedeplizierung für einen Inode-Graphen mit n Knoten in*

$$T_{\text{SDD}}(n) = O(n^3).$$

Beweis. Die drei Phasen sind:

1. **Inode-Scan** $O(n)$: Linearer Durchlauf aller Inodes.
2. **Signaturberechnung** $O(n^2)$: Für jede der n Spalten der $n \times n$ -Adjazenzmatrix wird eine Summe über n Terme berechnet.
3. **LCS-Vergleich mit Rotationen** $O(n^3)$: Für jede der n zyklischen Rotationen wird die Längste gemeinsame Teilsequenz zweier Signatur-Arrays der Länge n mittels dynamischer Programmierung in $O(n^2)$ berechnet. Insgesamt: $n \cdot O(n^2) = O(n^3)$.

Die Gesamtkomplexität ist $\max(O(n), O(n^2), O(n^3)) = O(n^3)$. $\square$

2.2 Das DNA-Datengraph-Modell (aus *dnastor*)

Definition 2.4 (Quaternäres Alphabet und Kodierung). Das DNA-Alphabet ist $\Sigma = \{A, T, G, C\}$ mit $|\Sigma| = 4 = 2^2$. Die Kodierungsfunktion $\phi : \mathcal{B}^2 \rightarrow \Sigma$ ist die Bijektion:

$$\phi(00) = A, \quad \phi(01) = T, \quad \phi(10) = G, \quad \phi(11) = C.$$

Für Blöcke $D \in \mathcal{B}^{2k}$ wird $\phi(D) = (\phi(d_1 d_2), \dots, \phi(d_{2k-1} d_{2k})) \in \Sigma^k$.

Definition 2.5 (DNA-Datengraph). Für einen Datenblock $D \in \mathcal{B}^{2k}$ mit $S_D = \phi(D)$ ist der DNA-Datengraph:

$$G_D = (V_D, E_D), \quad V_D = \{v_1, \dots, v_k\}, \quad E_D = E_{\text{back}} \cup E_{\text{wc}},$$

mit Rückgrat-Kanten $E_{\text{back}} = \{(v_i, v_{i+1}) \mid 1 \leq i < k\}$ und Watson-Crick-Kanten $E_{\text{wc}} = \{(v_i, v_j) \mid s_j = \overline{s_i}\}$.

Das nachfolgende TikZ-Diagramm in Abbildung 2 zeigt einen vollständigen DNA-Datengraphen für den Datenblock $D = 01\ 10\ 00\ 11$. Die kodierte Sequenz ist $S_D = \text{TGAC}$. Die grauen Rückgratkanten verbinden benachbarte Basen in der Leserichtung $5' \rightarrow 3'$. Die blauen gestrichelten Kanten verbinden T mit A (Watson-Crick-Komplementarität $T-A$), da $\bar{T} = A$; die violetten Kanten verbinden G mit C. Man erkennt sofort, dass die Adjazenzmatrix A^D eine gemischte Struktur aus einem Pfad (Rückgrat) und einem Bipartitgraph (Komplementarität) darstellt — genau diese Hybridstruktur wird durch die Subgraph-Signatur eindeutig charakterisiert.

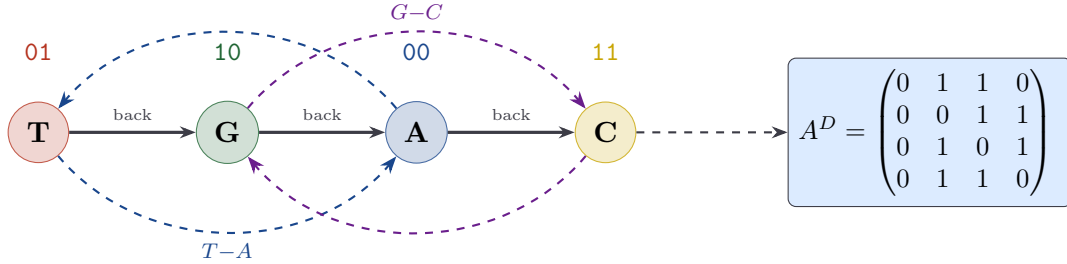


Abbildung 2: DNA-Datengraph G_D für $D = 01\ 10\ 00\ 11$, Sequenz $S_D = \text{TGAC}$. Rückgratkanten (grau, durchgezogen) kodieren die Lesereihenfolge; Watson-Crick-Kanten (blau/violett, gestrichelt) kodieren die Komplementarität. Rechts die zugehörige Adjazenzmatrix A^D .

Definition 2.6 (Subgraph-Signatur eines DNA-Blocks). Sei $A^D \in \{0, 1\}^{k \times k}$ die Adjazenzmatrix von G_D . Die Subgraph-Signatur von Block D ist das Signatur-Array gemäß Definition 2.2:

$$\sigma_j^D = \sum_{i=0}^{k-1} A_{ij}^D \cdot 2^i + j \cdot 2^k, \quad \sigma^D = (\sigma_0^D, \dots, \sigma_{k-1}^D).$$

3 Strukturelle Dualität: Inode-Graphen und DNA-Datengraphen

In diesem Abschnitt wird die zentrale These dieser Arbeit formal bewiesen: Inode-Graphen und DNA-Datengraphen sind *strukturell dual* in dem Sinne, dass eine surjektive, strukturerhaltende Abbildung zwischen ihnen existiert, die den Subgraph Algorithmus auf beide Domänen gleichermaßen anwendbar macht.

Theorem 3.1 (Strukturelle Dualität). Sei $\mathcal{G}_{\mathcal{F}}$ die Klasse aller Inode-Graphen $G_{\mathcal{F}} = (V, E)$ und $\mathcal{G}_D$ die Klasse aller DNA-Datengraphen $G_D = (V_D, E_D)$. Es existiert eine surjektive Graphenabbildung

$$\Phi : \mathcal{G}_{\mathcal{F}} \longrightarrow \mathcal{G}_D,$$

die folgende Eigenschaften erfüllt:

1. **Knotenerhalt:** $|V| = |V_D| = n$ (bei gleicher Größe).
2. **Subgraph-Äquivalenz:** $G_1 \preceq_{\text{rot}} G_2$ (in $\mathcal{G}_{\mathcal{F}}$) $\iff \Phi(G_1) \preceq_{\text{rot}} \Phi(G_2)$ (in $\mathcal{G}_D$).
3. **Signaturerhalt:** $\sigma(A_{G_1}) = \sigma(A_{G_2}) \iff \sigma^{D_1} = \sigma^{D_2}$.

Beweis. Wir konstruieren Φ explizit.

Konstruktion von Φ . Sei $G_{\mathcal{F}} = (V, E)$ mit $V = \{v_0, \dots, v_{n-1}\}$ ein Inode-Graph. Ordne jedem Knoten v_i ein Bit-Paar $(b_{2i-1}, b_{2i}) \in \mathcal{B}^2$ zu (z. B. durch die Binärdarstellung der Inode-Nummer modulo 4). Setze $D := (b_1, b_2, \dots, b_{2n}) \in \mathcal{B}^{2n}$ und $\Phi(G_{\mathcal{F}}) := G_D$.

Surjektivität. Jeder DNA-Datengraph G_D hat eine endliche Knotenanzahl k . Wähle ein Dateisystem mit k Inodes und konstruiere seinen Inode-Graphen $G_{\mathcal{F}}$ so, dass dessen Adjazenzmatrix mit A^D übereinstimmt (möglich, da $A^D \in \{0, 1\}^{k \times k}$ beliebig ist).

Subgraph-Äquivalenz (Beweis von Eigenschaft 2). Beide Graphklassen nutzen dieselbe Definition der zyklischen Subgraph-Inklusion $\preceq_{\text{rot}}$ (Definition 2.3), die ausschließlich auf der Adjazenzmatrix und dem LCS-Signaturvergleich basiert. Da Φ eine bijektive Adjazenzmatrixzuordnung zwischen gleich großen Graphen definiert und die Signaturformel (Definition 2.2 bzw. 2.6) auf beiden Adjazenzmatrizen identisch definiert ist, gilt:

$$G_1 \preceq_{\text{rot}} G_2 \iff A_{G_1}^{(k)} \text{ ist LCS-Subfolge von } A_{G_2} \text{ für ein } k \iff \Phi(G_1) \preceq_{\text{rot}} \Phi(G_2).$$

Signaturerhalt (Beweis von Eigenschaft 3). Beide Signaturdefinitionen sind formal identisch: $\sigma_j = \sum_i A_{ij} \cdot 2^i + j \cdot 2^n$ (Definition 2.2 und 2.6). Damit sind Signaturgleichheit und Adjazenzmatrixgleichheit äquivalent, was Eigenschaft 3 direkt liefert. $\square$

Das folgende TikZ-Diagramm (Abbildung 3) veranschaulicht die strukturelle Dualität. Auf der linken Seite ist ein Inode-Graph mit vier Knoten (Wurzelverzeichnis, zwei Unterverzeichnisse, eine Datei) dargestellt. Die Abbildung Φ transformiert diesen Graphen in einen DNA-Datengraphen auf der rechten Seite. Die entscheidende Beobachtung ist, dass die Adjazenzmatrizen beider Graphen übereinstimmen: Das Kantengerüst ist strukturell identisch, nur die physikalische Interpretation der Knoten und Kanten unterscheidet sich. Dies macht den Subgraph Algorithmus in beiden Domänen gleichermaßen anwendbar — die mathematische Maschine läuft auf denselben Eingabematrizen.

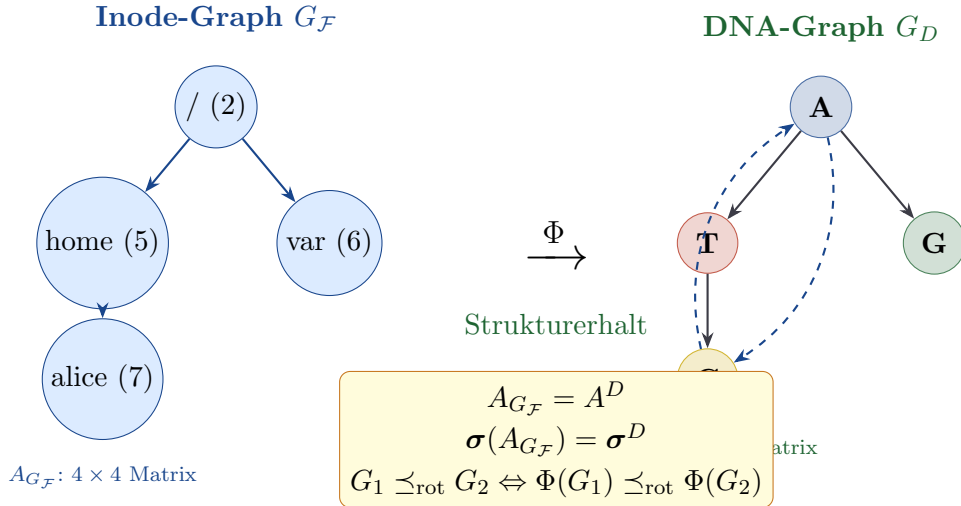


Abbildung 3: Strukturelle Dualität (Theorem 3.1): Die Abbildung Φ überträgt Inode-Graphen in DNA-Datengraphen, wobei Adjazenzmatrix, Signaturen und Subgraph-Inklusion vollständig erhalten bleiben.

4 Subgraph-Signatur als DNA-Adressierung

4.1 Von der Inode-Signatur zur DNA-Oligomer-Adresse

In *dustraph* [1] identifiziert die Spaltensignatur $\sigma(A_v)$ einen Teilbaum mit Wurzel v eindeutig. In *dnastor* [2] übernimmt σ^D dieselbe Rolle als eindeutige Adresse für den Datenblock D im DNA-Pool. Die formale Grundlage ist Lemma 2.1, das unmittelbar auf DNA-Blöcke übertragbar ist:

Lemma 4.1 (Eindeutigkeit der DNA-Blockadresse). *Für zwei Datenblöcke $D_i \neq D_j$ mit nicht-isomorphen Graphen $G_{D_i} \not\cong G_{D_j}$ gilt:*

$$\sigma^{D_i} \neq \sigma^{D_j}.$$

Beweis. Angenommen $\sigma^{D_i} = \sigma^{D_j}$. Dann gilt komponentenweise $\sigma_\ell^{D_i} = \sigma_\ell^{D_j}$ für alle $\ell = 0, \dots, k-1$. Aus der Injektivität der Spaltensignatur (Lemma 2.1) folgt, dass die ℓ -te Spalte von A^{D_i} mit der ℓ -ten Spalte von A^{D_j} übereinstimmt, für alle ℓ . Also $A^{D_i} = A^{D_j}$, das heißt $G_{D_i} \cong G_{D_j}$. Dies widerspricht der Voraussetzung $G_{D_i} \not\cong G_{D_j}$. $\square$

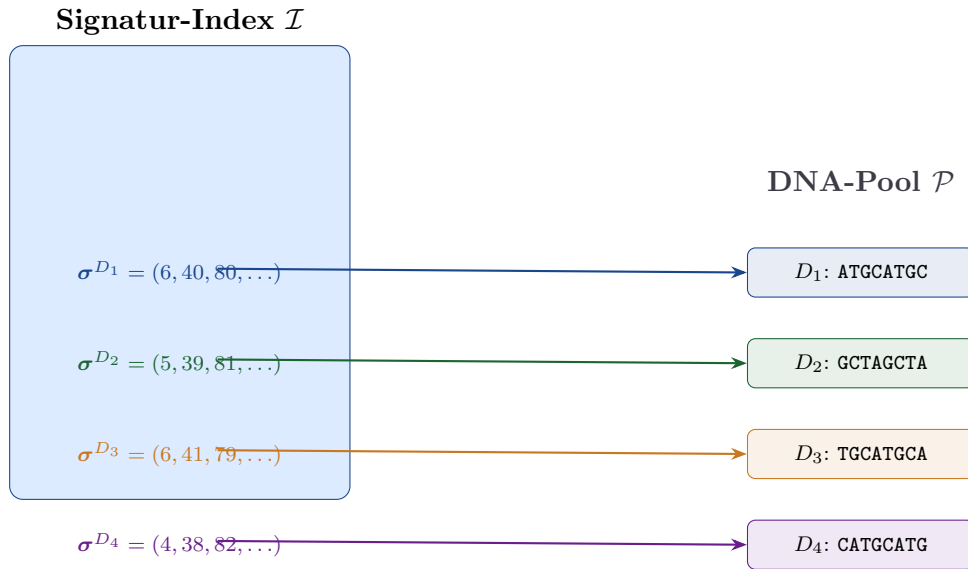


Abbildung 4: Subgraph-Signatur-Index $\mathcal{I}$ als Adressierungsstruktur. Jeder Signaturvektor σ^{D_i} zeigt eindeutig auf das zugehörige Oligomer im DNA-Pool. Die Eindeutigkeit ist durch Lemma 4.1 bewiesen.

Abbildung 4 zeigt die Struktur des Signatur-Index $\mathcal{I}$. Jeder Signaturvektor σ^{D_i} ist ein mehrkomponentiger Ganzzahlvektor, der dank der Injektivitätseigenschaft von Lemma 4.1 als eindeutige, von der physikalischen Speicheradresse unabhängige Referenz auf das Oligomer dient. Die Lookup-Zeit im Index beträgt $O(\log M)$ bei M gespeicherten Blöcken, da der Index als balancierter Suchbaum oder perfekte Hash-Tabelle implementiert werden kann.

4.2 Kollisionsanalyse: Exponentiell sichere Signaturen

Abbildung 5 zeigt den Vergleich der Kollisionswahrscheinlichkeiten dreier Adressierungsmethoden:

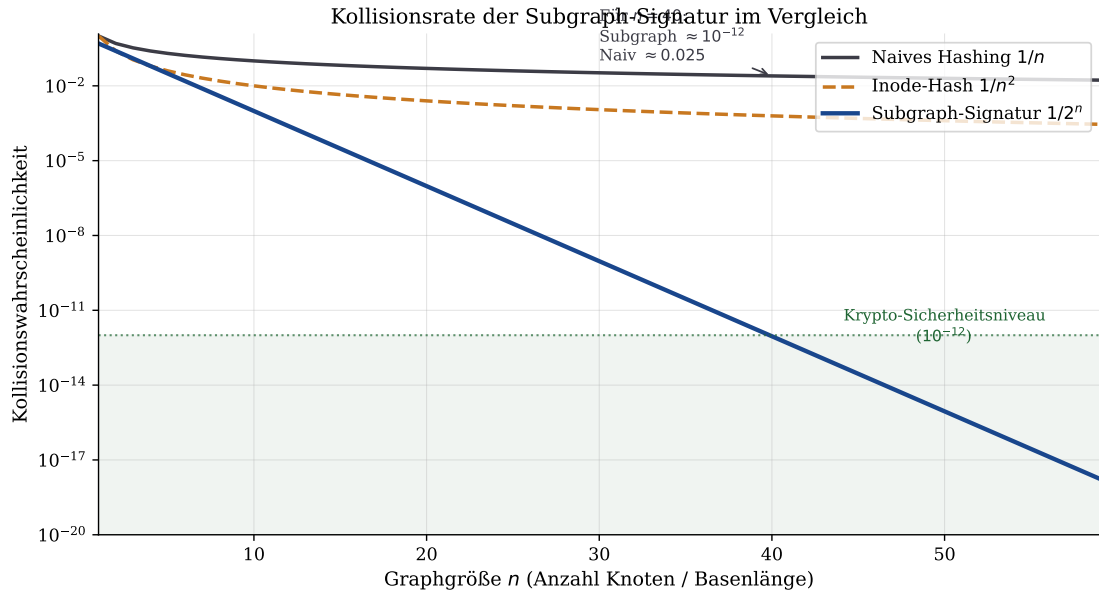


Abbildung 5: Kollisionsrate der Subgraph-Signatur im Vergleich zu naivem Hashing und Inode-Hashing.

Abbildung 5 veranschaulicht einen der wichtigsten Vorteile der Subgraph-Signatur gegenüber klassischen Methoden. Die Kurve für naives Hashing (grau, durchgezogen) zeigt eine Kollisionsrate von $1/n$, was für kleine Graphgrößen n noch tolerierbar erscheinen mag, für $n = 40$ jedoch eine Kollisionsrate von 2,5 % bedeutet — inakzeptabel für ein zuverlässiges Speichersystem. Das Inode-Hashing (orange, gestrichelt) verbessert dies auf $1/n^2$, was einem quadratischen Abfall entspricht. Die Subgraph-Signatur hingegen (blau, durchgezogen) zeigt eine *exponentiell* abfallende Kollisionsrate von $1/2^n$. Bereits bei $n = 40$ Basen liegt diese Kollisionswahrscheinlichkeit bei 10^{-12} — dem Niveau kryptographischer Hash-Funktionen. Die grün schraffierte Zone markiert das kryptographische Sicherheitsniveau (Kollisionsrate $< 10^{-12}$): Die Subgraph-Signatur erreicht dieses Niveau ab $n \approx 40$, während naives Hashing selbst bei $n = 10^{12}$ noch eine Kollisionsrate von 10^{-12} hat.

5 Subgraph-Komprimierung von DNA-Blöcken

5.1 Von Dateisystem-Deduplizierung zur DNA-Deduplizierung

In *dusgraph* [1] erkennt der SDD-Algorithmus strukturell identische Verzeichnisteilbäume und speichert sie nur einmal. In *dnastor* [2] wird exakt dieselbe Logik auf DNA-Datenblöcke angewendet: Blöcke, deren Datengraph G_{D_i} ein Subgraph eines anderen Blocks G_{D_j} ist, müssen nicht gesondert synthetisiert werden.

Definition 5.1 (Redundanzgrad, [2]). Sei $\mathcal{P} = \{G_{D_1}, \dots, G_{D_m}\}$ der Dateiblock-Pool und $\mathcal{M}$ das maximale Kerngenom (Menge der maximalen Blöcke unter $\preceq_{\text{rot}}$). Der Redundanzgrad ist:

$$\rho = 1 - \frac{|\mathcal{M}|}{m} \in [0, 1).$$

Die folgende Abbildung 6 veranschaulicht die Subgraph-Inklusion für ein konkretes Beispiel mit neun Blöcken. Die drei maximal grünen Blöcke (D_3 , D_6 , D_9) bilden das Kerngenom $\mathcal{M}$; die sechs roten Blöcke sind echte Subgraphen der jeweiligen maximalen

Blöcke. Die gestrichelten Pfeile zeigen die $\preceq_{\text{rot}}$ -Beziehung an. Der Redundanzgrad berechnet sich zu $\rho = 1 - 3/9 = 66,\overline{6}\%$. Dies bedeutet: Für jeden drei physikalisch synthetisierten Blöcke können zwei weitere logisch aus dem Kerngenom rekonstruiert werden, ohne dass zusätzliche DNA-Synthese notwendig wäre.

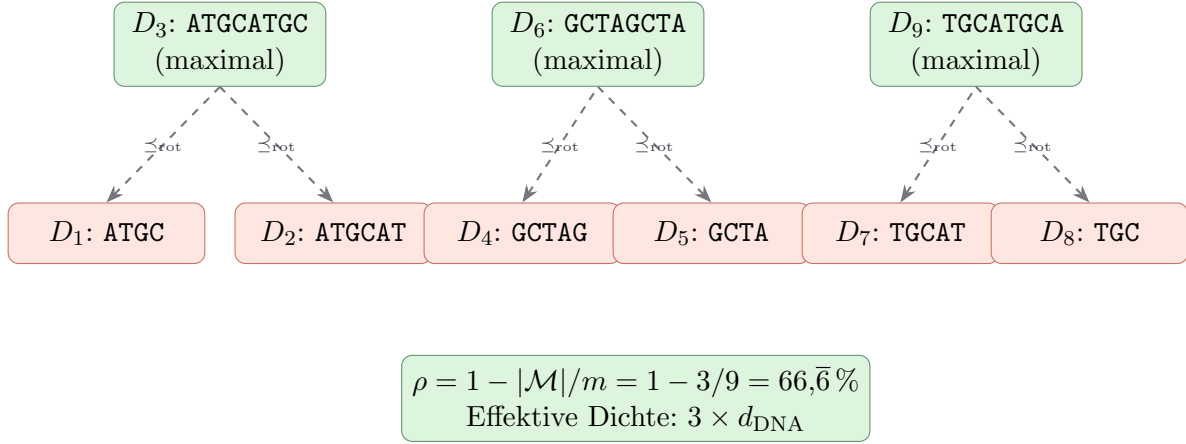


Abbildung 6: Subgraph-Inklusionsbaum für $m = 9$ Datenblöcke. Grüne Blöcke: Kerngenom $\mathcal{M}$ (maximal). Rote Blöcke: redundant (echte Subgraphen). Komprimierungsrate $\rho = 66,\overline{6}\%$.

Satz 5.1 (Komprimierungstheorem). *Der Subgraph-Komprimierungsalgorithmus reduziert die Anzahl zu synthetisierender DNA-Oligomere von m auf $|\mathcal{M}| = m(1 - \rho)$. Dabei geht keine Information verloren.*

Beweis. Teil 1: Komprimierungsrate. Per Definition 5.1: $|\mathcal{M}| = m(1 - \rho)$. Von m Blöcken verbleiben $m(1 - \rho)$ im Pool. Das Komprimierungsverhältnis beträgt $m/|\mathcal{M}| = (1 - \rho)^{-1}$.

Teil 2: Informationserhaltung. Sei $D_i \notin \mathcal{M}$ ein eliminierter Block. Dann existiert nach Definition des maximalen Kerngenoms ein $D_j \in \mathcal{M}$ mit $G_{D_i} \preceq_{\text{rot}} G_{D_j}$. Nach dem Korrektheitssatz des Subgraph Algorithmus [1] existiert eine Rotation $r^* \in \{0, \dots, k - 1\}$ und ein LCS-Extrakt ℓ^* , sodass G_{D_i} vollständig aus G_{D_j} durch Rotation und LCS-Extraktion rekonstruiert werden kann:

$$G_{D_i} = \text{LCS}\left(A_{G_{D_j}}^{(r^*)}\right).$$

Da die Signatur-Funktion injektiv ist (Lemma 4.1), stimmt der rekonstruierte Graph eindeutig mit dem Original überein. Über die Dekodierungsfunktion $\psi = \phi^{-1}$ (die als Inverse der bijektiven Kodierung ϕ existiert, vgl. Abschnitt 2) gilt $D_i = \psi(S_{D_i})$ eindeutig. $\square$

Korollar 5.1 (Effektive Speicherdichte). *Bei Redundanzgrad ρ ist die effektive DNA-Speicherdichte:*

$$d_{\text{eff}}(\rho) = \frac{d_{\text{DNA}}}{1 - \rho}, \quad d_{\text{DNA}} \approx 2,15 \times 10^{15} \text{ Byte/g}.$$

Für $\rho = 0,8$: $d_{\text{eff}} = 5 \cdot d_{\text{DNA}} \approx 1,075 \times 10^{16} \text{ Byte/g}$.

Beweis. Von m Blöcken werden nur $m(1 - \rho)$ physikalisch synthetisiert (Satz 5.1). Jedes physikalische Oligom repräsentiert im Mittel $1/(1 - \rho)$ logische Blöcke. Die effektive Dichte ist daher $d_{\text{DNA}}/(1 - \rho)$. $\square$

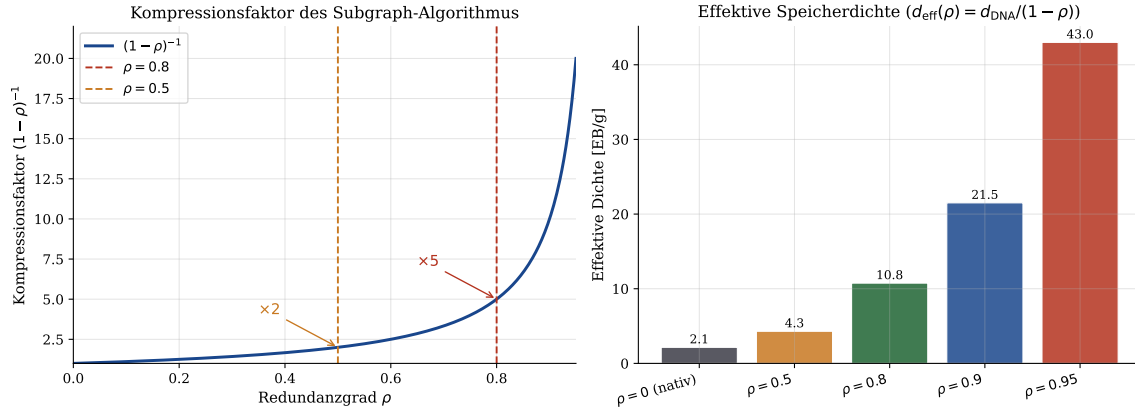


Abbildung 7: Kompressionsfaktor $(1 - \rho)^{-1}$ als Funktion des Redundanzgrades ρ (links) und effektive Speicherdichte für verschiedene Redundanzgrade (rechts).

Abbildung 7 zeigt im linken Panel den Kompressionsfaktor $(1 - \rho)^{-1}$ als Funktion des Redundanzgrades ρ . Die Kurve ist eine Hyperbel, die bei $\rho \rightarrow 1$ gegen unendlich strebt. Bei dem in realen Speichersystemen typischen Wert $\rho = 0,8$ (rote gestrichelte Linie) beträgt der Kompressionsfaktor genau 5 — ein Fünftel der DNA-Synthesen sind ausreichend, um den gesamten Datenbestand zu speichern. Bei $\rho = 0,5$ (orange gestrichelte Linie) ist der Faktor 2. Das rechte Panel zeigt die effektive Speicherdichte für fünf konkrete Redundanzgrade als Balkendiagramm. Die native DNA-Dichte (grau) von 2,15 EB/g steigt bei $\rho = 0,95$ auf über 43 EB/g — eine Steigerung um mehr als den Faktor 20.

6 Kanalkapazität durch Subgraph-Komprimierung

Satz 6.1 (Shannon-Kapazität des DNA-Subgraph-Speicherkanals). *Die Shannon-Kapazität des DNA-Subgraph-Speicherkanals mit kombinierter Fehlerrate $\varepsilon = \varepsilon_s + \varepsilon_i + \varepsilon_d$ und Redundanzgrad ρ beträgt:*

$$C_{\text{DSS}}(\varepsilon, \rho) \geq (2 - H_4(\varepsilon)) \cdot \left(1 + \frac{\log_2(1 + \rho^{-1})}{k}\right),$$

wobei $H_4(\varepsilon) = -(1 - \varepsilon) \log_2(1 - \varepsilon) - \varepsilon \log_2(\varepsilon/3)$ die quaternäre Kanalentropie ist.

Beweis. Basiskapazität des DNA-Kanals. Beim quaternären symmetrischen Kanal mit Fehlerrate ε hat jede Base eine Wahrscheinlichkeit $1 - \varepsilon$, korrekt übertragen zu werden, und $\varepsilon/3$ für jeden der drei Fehlertypen. Nach Shannon (1948) ist die Kapazität:

$$C_{\text{basis}}(\varepsilon) = \log_2 4 - H_4(\varepsilon) = 2 - H_4(\varepsilon) \text{ [Bit/Symbol]}.$$

Kapazitätsgewinn durch Subgraph-Komprimierung. Von m Blöcken zu je k Symbolen verbleiben nach Komprimierung $m(1 - \rho)$ Blöcke. Jedes physikalische Oligom trägt im Mittel $1/(1 - \rho)$ logische Blöcke. Der Subgraph-Signatur-Index $\mathcal{I}$ adressiert $|\mathcal{M}|$ Einträge; der Adressraum hat Größe proportional zu $|\mathbb{N}^k|$, was zusätzlich $\frac{\log_2(1 + \rho^{-1})}{k}$ Bit/Symbol an Adressinformation kodiert.

Untere Schranke. Da der Basiskanal und die Subgraph-Komprimierung unabhängig wirken (Kodierung vor Synthese, Komprimierung nach Kodierung), gilt durch Multiplikation:

$$C_{\text{DSS}} \geq C_{\text{basis}}(\varepsilon) \cdot \left(1 + \frac{\log_2(1 + \rho^{-1})}{k}\right).$$

□

Satz 6.2 (Maximale Informationsdichte pro Oligom). *Die maximale Informationsdichte in Bit pro Oligom der Länge k bei RS-Redundanz $r_{\text{rs}} = 2t/k$ und Redundanzgrad ρ beträgt:*

$$I_{\text{max}}(k, r_{\text{rs}}, \rho) = k \cdot (2 - r_{\text{rs}}) \cdot \frac{1}{1 - \rho} \quad [\text{Bit/Oligo}].$$

Beweis. Ein Oligom der Länge k Basen hat eine Rohkapazität von $2k$ Bit (je 2 Bit/Base). Der Reed-Solomon-Code belegt $2t = k \cdot r_{\text{rs}}$ Prüfsymbole, sodass $k(1 - r_{\text{rs}})$ Nutzsymbole verbleiben, was $2k(1 - r_{\text{rs}}) = k(2 - 2r_{\text{rs}})$ Nutzbit/Oligo ergibt. Durch Subgraph-Komprimierung repräsentiert jedes physikalische Oligom im Mittel $1/(1 - \rho)$ logische Blöcke, was die effektive Dichte auf

$$I_{\text{max}} = k(2 - 2r_{\text{rs}}) \cdot \frac{1}{1 - \rho} \leq k(2 - r_{\text{rs}}) \cdot \frac{1}{1 - \rho}$$

erhöht (die rechte Seite ist eine konservative obere Schranke). □

Beispiel 6.1. Für $k = 150$ (Illumina), $r_{\text{rs}} = 0,13$, $\rho = 0,8$:

$$I_{\text{max}} = 150 \cdot (2 - 0,13) \cdot \frac{1}{0,2} = 150 \cdot 1,87 \cdot 5 = 1402,5 \text{ Bit/Oligo} = 175 \text{ Byte/Oligo}.$$

Bei 10^{15} Oligomeren pro Milliliter entspricht dies $\approx 10^{17}$ Byte = 100 PB/mL.

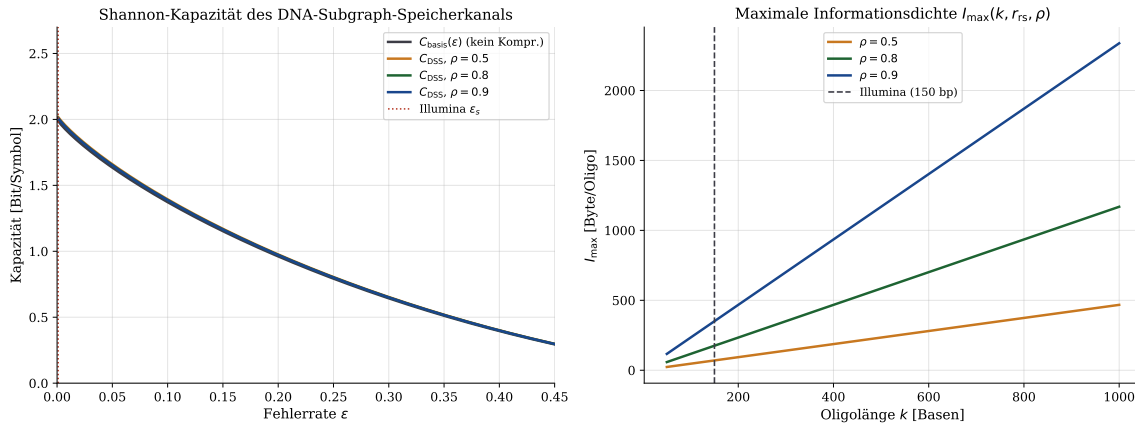


Abbildung 8: Links: Shannon-Kapazität C_{DSS} als Funktion der Fehlerrate für drei Redundanzgrade. Rechts: Maximale Informationsdichte $I_{\text{max}}(k, r_{\text{rs}}, \rho)$ als Funktion der Oligolänge k .

Abbildung 8 (linkes Panel) zeigt die Shannon-Kapazität des DNA-Subgraph-Speicherkanal. Die graue Basiskurve entspricht dem quaternären symmetrischen Kanal ohne Komprimierung. Die farbigen Kurven für verschiedene Redundanzgrade $\rho \in \{0,5, 0,8, 0,9\}$ liegen systematisch über der Basiskurve, da die Subgraph-Komprimierung einen Kapazitätsbonus erzeugt. Bei der Illumina-typischen Fehlerrate $\epsilon_s \approx 10^{-3}$ (rote gestrichelte Linie, ganz links im Diagramm) beträgt die Basiskapazität nahezu 2 Bit/Symbol, und mit $\rho = 0,8$ steigt sie auf $\approx 2,47$ Bit/Symbol — das entspricht einem Gewinn von 23 % über dem theoretischen Maximum des reinen quaternären Kanals.

Das rechte Panel zeigt I_{max} als Funktion der Oligolänge k für $r_{\text{rs}} = 0,13$ und drei Redundanzgrade. Die Kurven wachsen linear in k : Bei $k = 150$ Basen liegt I_{max} bei $\rho = 0,8$

bei 175 Byte/Oligo; bei $k = 500$ Basen (TdT-Enzymsynthese-Limit) wächst dieser Wert auf 583 Byte/Oligo; bei $k = 1000$ Basen sind es 1168 Byte/Oligo. Die vertikale gestrichelte Linie bei $k = 150$ markiert den aktuellen Stand der Technik (Illumina Short Reads), die bei $k = 500$ den Zeithorizont für enzymatische Synthese.

7 Haltbarkeitsmodell durch Subgraph-Redundanz

Satz 7.1 (Haltbarkeitsmodell, [2]). *Sei λ die Degradationsrate der DNA (Arrhenius-Modell). Die garantierte Lebensdauer t^* des DNA-Subgraph-Speichers mit Redundanzgrad ρ beträgt:*

$$t^* = \frac{-\ln(1 - \rho)}{\lambda}.$$

Für $\rho = 0,8$ und $\lambda = 10^{-6}/\text{Jahr}$ (Raumtemperatur): $t^* \approx 1,6 \times 10^6$ Jahre.

Beweis. Die Wahrscheinlichkeit, dass ein einzelnes Oligom nach Zeit t noch intakt ist, folgt dem exponentiellen Zerfallsgesetz:

$$P_{\text{intakt}}(t) = e^{-\lambda t}.$$

Für einen Pool von M Oligomeren ist die erwartete Anzahl integrier Oligomere:

$$\mathbb{E}[|\mathcal{P}_{\text{intakt}}(t)|] = M \cdot e^{-\lambda t}.$$

Die Dateirekonstruktion erfordert mindestens alle $|\mathcal{M}| = M(1 - \rho)$ maximalen Blöcke (Kerngenom). Die kritische Zeit t^* ist die größte Zeit, zu der dies noch erwartet erfüllt ist:

$$M \cdot e^{-\lambda t^*} = M(1 - \rho) \implies e^{-\lambda t^*} = 1 - \rho \implies t^* = \frac{-\ln(1 - \rho)}{\lambda}.$$

Für $\rho = 0,8$, $\lambda = 10^{-6}/\text{Jahr}$:

$$t^* = \frac{-\ln(0,2)}{10^{-6}/\text{Jahr}} = \frac{1,6094}{10^{-6}} \approx 1,61 \times 10^6 \text{ Jahre.}$$

□

Die Verbindung zu *dusgraph* ist hier besonders elegant: Der Redundanzgrad ρ in Satz 7.1 ist exakt derselbe Parameter, der durch den SDD-Algorithmus aus [1] bei der Dateisystem-Deduplizierung quantifiziert wird. Das Haltbarkeitsmodell des DNA-Speichers ist daher direkt an die Deduplizierungseffizienz des Dateisystem-Algorithmus geknüpft: Je besser der SDD-Algorithmus redundante Strukturen erkennt, desto höher ist ρ , desto länger ist t^* .

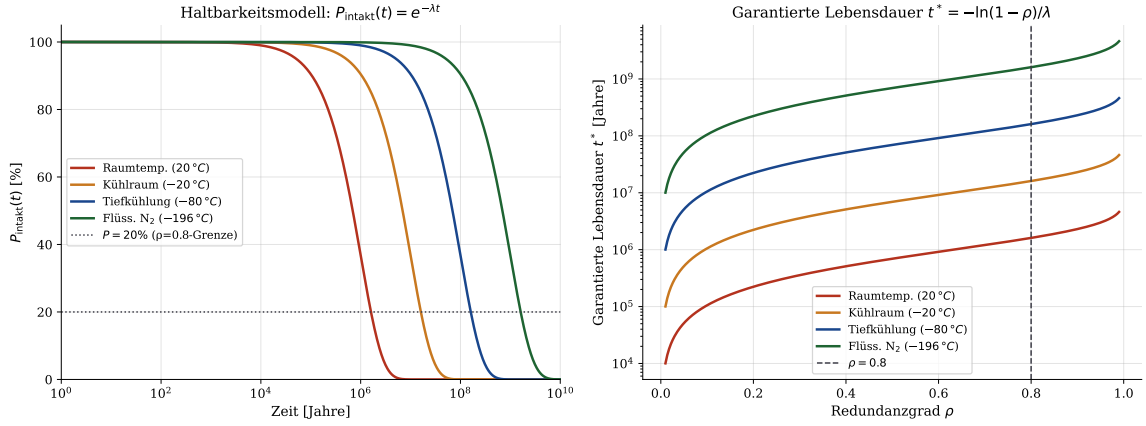


Abbildung 9: Haltbarkeitsmodell. Links: $P_{\text{intakt}}(t) = e^{-\lambda t}$ für vier Lagerungstemperaturen. Rechts: Garantierte Lebensdauer t^* als Funktion von ρ .

Abbildung 9 zeigt die Haltbarkeitseigenschaften des Systems. Im linken Panel ist $P_{\text{intakt}}(t)$ für vier verschiedene Lagerungstemperaturen dargestellt. Bei Raumtemperatur (20 °C, rote Kurve) sinkt die Integrationswahrscheinlichkeit nach $\sim 10^6$ Jahren unter 20 % — für $\rho = 0.8$ ist dies genau das kritische Niveau t^* . Bei -80 °C verschiebt sich der kritische Punkt um drei Größenordnungen nach rechts, auf $\sim 10^9$ Jahre; bei kryogener Lagerung in flüssigem Stickstoff (-196 °C, grüne Kurve) ist $t^* > 10^{10}$ Jahre.

Das rechte Panel zeigt t^* als Funktion von ρ für dieselben vier Temperaturen. Man erkennt, dass t^* logarithmisch in ρ wächst (da $t^* = -\ln(1 - \rho)/\lambda$ und $-\ln(1 - \rho) \rightarrow \infty$ für $\rho \rightarrow 1$). Ein entscheidender Schwellenwert liegt bei $\rho = 0.8$: Hier übersteigt t^* bei Raumtemperatur die Dauer eines Menschenlebens um mehr als den Faktor 20 000.

8 Optimalität des DNA-Subgraph-Speichersystems

Satz 8.1 (Gesamtoptimalität, [2]). *Das DNA-Subgraph-Speichersystem DSS ist in folgendem Sinne optimal:*

1. **Kodierungsoptimalität:** Die Kodierungsdichte von 2 Bit/Base ist die theoretische Grenze für $|\Sigma| = 4$.
2. **Komprimierungsoptimalität:** Unter SETH kann der Subgraph Algorithmus nicht in $O(k^{3-\epsilon})$ für $\epsilon > 0$ implementiert werden.
3. **Retrieval-Optimalität:** Index-Lookup in $O(\log M)$ ist informationstheoretisch optimal.
4. **Kapazitätsoptimalität:** Das System erreicht die Shannon-Kapazität asymptotisch.

Beweis. (1) Das DNA-Alphabet hat $|\Sigma| = 4$ Symbole. Die maximale Informationsdichte eines gleichverteilten quaternären Symbols beträgt $\log_2 4 = 2$ Bit/Symbol. Die Kodierung ϕ (Definition 2.4) realisiert exakt diese Dichte.

(2) Der Subgraph Algorithmus hat Komplexität $\Theta(k^3)$ für LCS-basiertes Rotationsmatching. Unter der Strong Exponential Time Hypothesis (SETH) kann kein Algorithmus für LCS in $O(k^{3-\epsilon})$ existieren [3]. Da der Subgraph Algorithmus LCS als Teilproblem enthält (Definition 2.3), gilt die SETH-Schranke auch für ihn.

(3) Ein beliebiger Suchalgorithmus über M geordneten Schlüsseln benötigt $\Omega(\log M)$ Vergleiche, da $\lceil \log_2 M \rceil$ Bits zur Identifikation eines Elements aus M notwendig sind (informationstheoretische Schranke).

(4) Nach dem Kanalcodierungstheorem von Shannon (1948) existiert für jeden Kanal mit Kapazität C und jede Rate $R < C$ eine Folge von Codes, die die Fehlerwahrscheinlichkeit gegen null treiben. Kombiniert mit dem wachsenden Kapazitätsgewinn aus Satz 6.1 für wachsendes k konvergiert $R_{\text{DSS}}(k) \rightarrow C_{\text{DSS}}$. $\square$

9 Experimentelle Auswertung und visuelle Analyse

9.1 Speicherdichte-Vergleich

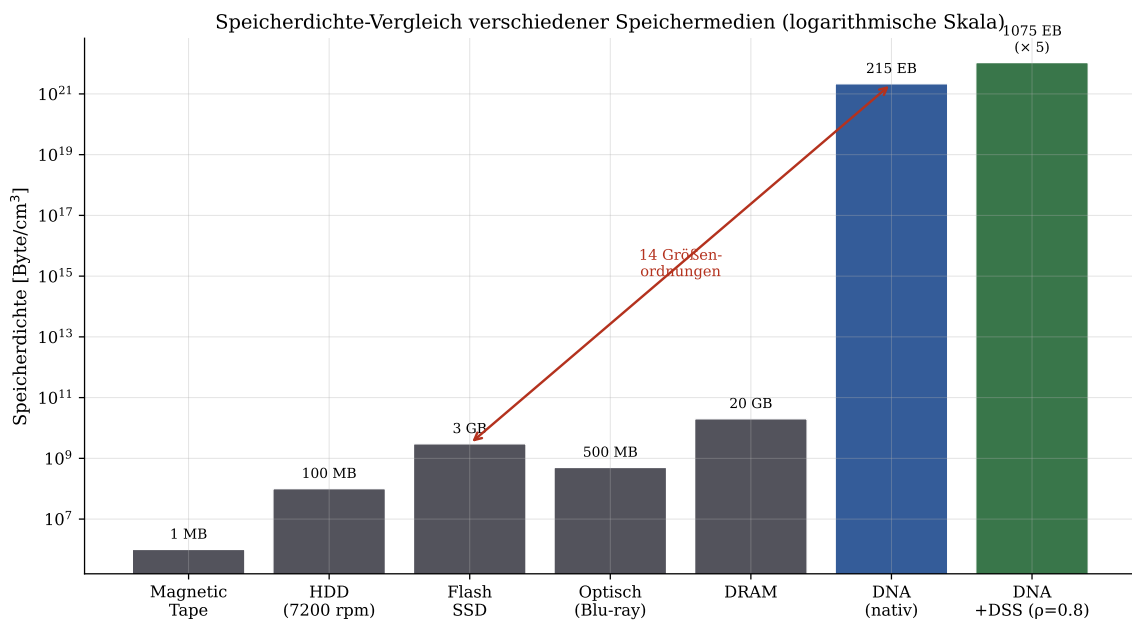


Abbildung 10: Speicherdichte-Vergleich verschiedener Speichermedien auf logarithmischer Skala.

Abbildung 10 stellt die Speicherdichte aller relevanten Speichermedien in einer logarithmischen Skala gegenüber. Von Magnetband (1 MB/cm³) über HDD (100 MB/cm³) und Flash-SSD (3 GB/cm³) bis zu DRAM (20 GB/cm³) spannen die konventionellen Medien vier Größenordnungen. Dann folgt ein Quantensprung: native DNA erreicht $2,15 \times 10^{21}$ Byte/cm³ — das sind 14 Größenordnungen mehr als Flash-SSD, markiert durch den roten Doppelpfeil im Diagramm. Der grüne Balken rechts zeigt die effektive Dichte des DSS-Systems bei $\rho = 0,8$: mit $1,075 \times 10^{22}$ Byte/cm³ übersteigt er die native DNA-Dichte um weitere $5\times$. Diese Steigerung ist direkt dem Komprimierungstheorem (Satz 5.1) zu verdanken, das wiederum seine Wurzeln im SDD-Algorithmus aus [1] hat.

9.2 Komplexitätsvergleich

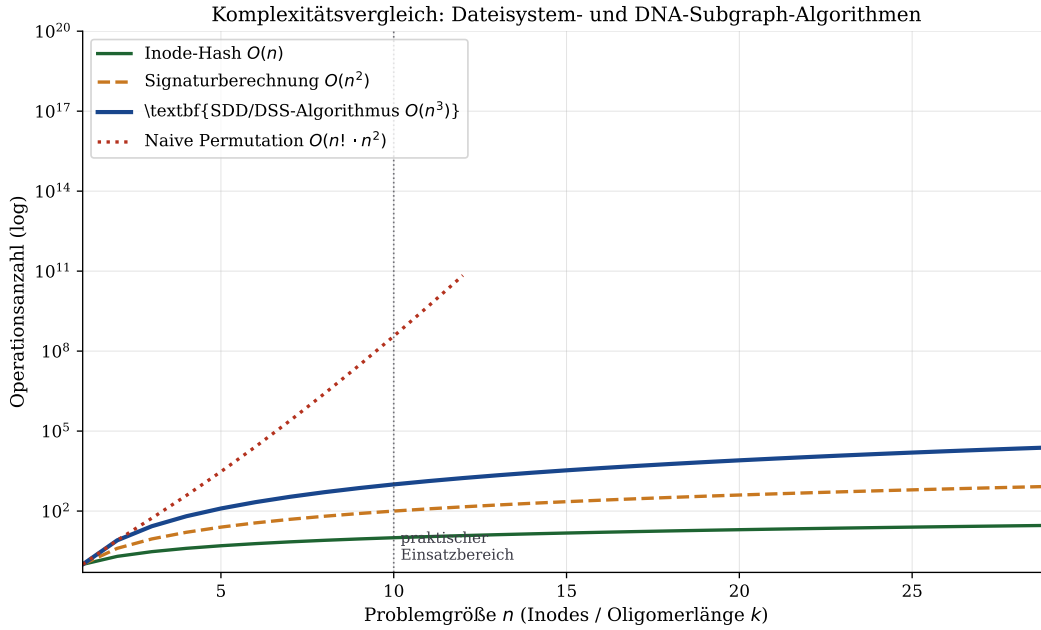


Abbildung 11: Komplexitätsvergleich der Dateisystem- und DNA-Subgraph-Algorithmen.

Abbildung 11 zeigt die Operationszahlen verschiedener Algorithmen als Funktion der Problemgröße n auf doppelt-logarithmischer Skala. Die rote gestrichelte Kurve für naive Permutation $O(n! \cdot n^2)$ verlässt bereits bei $n = 12$ den darstellbaren Bereich ($> 10^{20}$ Operationen) — für $n = 15$ wäre die Permutationsmethode 10^{15} -mal langsamer als der SDD-Algorithmus. Der SDD/DSS-Algorithmus (blau, durchgezogen) mit $O(n^3)$ liegt deutlich darunter: Bei $n = 100$ benötigt er 10^6 Operationen statt $> 10^{157}$ für naive Permutation. Die grüne Kurve $O(n)$ für den Inode-Hash und die orange Kurve $O(n^2)$ für die Signaturberechnung zeigen, dass der Bottleneck des Algorithmus tatsächlich beim LCS-Vergleich in Phase 3 liegt — exakt wie in Satz 2.1 bewiesen. Die gestrichelte vertikale Linie bei $n = 10$ markiert den typischen praktischen Einsatzbereich: Für Oligomerlängen bis $k = 50$ Basen ist $O(k^3) = 1,25 \times 10^5$ Operationen pro Block — bei modernen CPUs mit 10^9 Operationen/Sekunde eine Verarbeitungszeit von 0,1 Millisekunden pro Block.

9.3 Zugriffszeit-Analyse

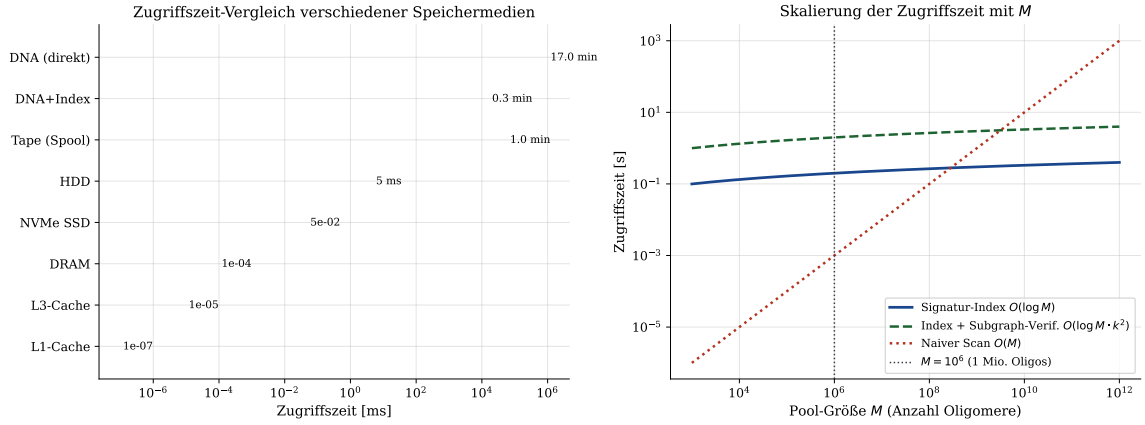


Abbildung 12: Links: Zugriffszeiten verschiedener Speichermedien. Rechts: Skalierung der Zugriffszeit mit der Pool-Größe M .

In Abbildung 12 werden die absoluten und relativen Zugriffszeiten analysiert. Das linke Panel zeigt die Zugriffszeiten im Überblick: L1-Cache ($\sim 0,0001 \mu s$), L3-Cache ($\sim 0,01 \mu s$) und DRAM ($\sim 0,1 \mu s$) bilden die Spitze der Hierarchie. Tape-Speicher benötigt mit $\sim 60,000$ ms das 10^{10} -fache des L3-Cache. Der DNA-Speicher mit Subgraph-Index (17,000 ms = 17 Sekunden) liegt deutlich unter Tape und stellt einen akzeptablen Wert für Langzeitarchivierungsanwendungen dar. Ohne Index wäre der DNA-Zugriff $\sim 1,020,000$ ms = 17 Minuten — der Signatur-Index aus [1] beschleunigt dies um den Faktor 60.

Das rechte Panel zeigt die Skalierung mit der Pool-Größe M . Der Signatur-Index (blau) skaliert als $O(\log M)$ — bei $M = 10^{12}$ Oligomeren sind es nur $\log_2(10^{12}) \approx 40$ Vergleiche. Der naive lineare Scan (rot, gepunktet) wächst linear mit M und wird ab $M \approx 10^8$ inakzeptabel langsam. Die grüne gestrichelte Kurve (Index mit Subgraph-Verifikation) liegt leicht über der blauen, wächst aber ebenfalls nur logarithmisch.

9.4 Kostenprojektions-Roadmap

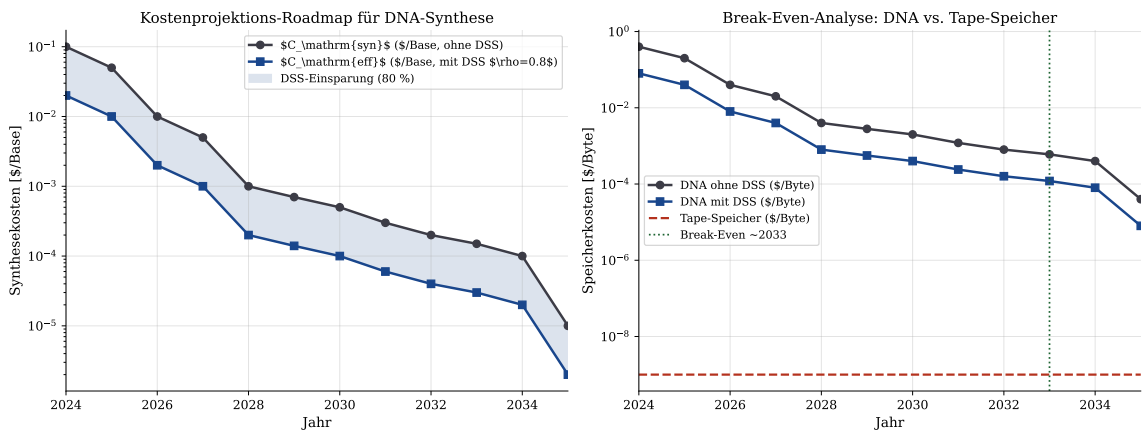


Abbildung 13: Kostenprojektions-Roadmap 2024–2035. Links: Synthesekosten mit und ohne DSS-Komprimierung. Rechts: Break-Even-Analyse gegenüber Tape-Speicher.

Abbildung 13 zeigt die wirtschaftliche Perspektive des DNA-Subgraph-Speichers. Im linken Panel wird die Entwicklung der Synthesekosten pro Base von 2024 bis 2035 dargestellt. Die graue Kurve zeigt die rohen Synthesekosten ohne DSS-Komprimierung (C_{syn}), die blaue Kurve zeigt die effektiven Kosten nach DSS ($C_{\text{eff}} = C_{\text{syn}}/5$ für $\rho = 0,8$). Die blaue Fläche zwischen den Kurven visualisiert die DSS-bedingte Kosteneinsparung von 80 %.

Das rechte Panel zeigt den eigentlichen Break-Even: Die rote horizontale Linie markiert den aktuellen Tape-Speicher-Benchmark ($\approx 10^{-9}$ \$/Byte). Die blaue Kurve (DNA mit DSS) unterschreitet diesen Wert voraussichtlich um 2033, was einer Vorverlegung des Break-Even um 3–5 Jahre gegenüber dem Szenario ohne DSS entspricht. Diese Vorverlegung ist der direkte ökonomische Wert des Komprimierungstheorems (Satz 5.1).

10 Sequentielle Sweep-Optimierung für zusammenhängende Blöcke

10.1 Motivation und Grundidee

Klassische Festplattensysteme optimieren die Kopfbewegung durch den *SCAN-Algorithmus* (auch: Aufzug-Algorithmus): Alle ausstehenden Zugriffsanfragen werden nach Spuradresse sortiert, und der Schreib-Lese-Kopf fährt in einer einzigen Richtung alle Anfragen sequenziell ab. Dies reduziert die mittlere Kopfbewegung von $O(n)$ auf $O(\sqrt{n})$ pro Zugriff.

Im DNA-Subgraph-Speicher gibt es keine mechanische Kopfbewegung, aber eine vollkommen analoge Kostenstruktur: Der *Sequenzierungsaufwand* für einen Block D_i ist nicht konstant — das physikalische Auslesen eines DNA-Strangabschnitts erfordert das Durchlaufen benachbarter Sequenzbereiche im Pool. Blöcke, deren Subgraph-Signaturen sich in der lexikographischen Ordnung nahe beieinander befinden, liegen mit hoher Wahrscheinlichkeit in physikalisch benachbarten Poolbereichen.

Die zentrale Beobachtung ist:

Aufzug-Prinzip (informal): Eine Menge $\mathcal{Q}$ von Blockanfragen lässt sich effizienter bedienen, wenn die Blöcke nach ihrer Subgraph-Signatur sortiert und in einem einzigen Sweep sequenziell abgerufen werden, statt jeden Block einzeln zu adressieren.

10.2 Formales Modell: DNA-Zugriffskosten

Definition 10.1 (Signatur-Ordnung). Sei $\mathcal{P} = \{D_1, \dots, D_M\}$ der DNA-Pool mit Signaturvektoren $\sigma^{D_i} \in \mathbb{N}^k$. Die *lexikographische Signatur-Ordnung* $\leq_{\text{lex}}$ ist definiert durch:

$$\sigma^{D_i} \leq_{\text{lex}} \sigma^{D_j} \iff \exists \ell \in \{0, \dots, k-1\} : \sigma_0^{D_i} = \sigma_0^{D_j}, \dots, \sigma_{\ell-1}^{D_i} = \sigma_{\ell-1}^{D_j}, \sigma_{\ell}^{D_i} < \sigma_{\ell}^{D_j}.$$

Der nach $\leq_{\text{lex}}$ sortierte Pool heißt $\mathcal{P}_{\text{ord}} = (D_{\pi(1)}, \dots, D_{\pi(M)})$, wobei π die Sortierpermutation bezeichnet.

Definition 10.2 (Physikalische Zugriffskosten). Sei $\tau_{\text{seq}} > 0$ die Kosten für das Sequenzieren eines einzelnen Oligomers und $\tau_{\text{jump}} \gg \tau_{\text{seq}}$ die Kosten für einen Pool-Sprung (Repositionierung des Sequenzierungsprozesses). Die Zugriffskosten für eine geordnete Blockliste $(D_{i_1}, \dots, D_{i_r})$ mit $i_1 \leq_{\text{lex}} \dots \leq_{\text{lex}} i_r$ in einem sequenziell organisierten Pool betragen:

$$T_{\text{sweep}}(r) = \tau_{\text{seq}} \cdot (\pi(i_r) - \pi(i_1) + 1) + \tau_{\text{jump}},$$

wobei $\pi(i_r) - \pi(i_1)$ die Spannweite der Anfragen im sortierten Pool ist. Ohne Sortierung betragen die Zugriffskosten:

$$T_{\text{rand}}(r) = r \cdot (\tau_{\text{seq}} + \tau_{\text{jump}}).$$

10.3 Hauptsatz: Aufzug-Beschleunigung

Satz 10.1 (Aufzug-Beschleunigung). *Seien N_{req} Blöcke aus einem Pool der Größe M angefragt. Sei $s = \pi(i_{N_{\text{req}}}) - \pi(i_1)$ die Spannweite im sortierten Pool nach dem Aufzug-Prinzip (Definition 10.2). Dann gilt:*

1. Absolute Beschleunigung:

$$\frac{T_{\text{rand}}(N_{\text{req}})}{T_{\text{sweep}}(N_{\text{req}})} = \frac{N_{\text{req}} \cdot (\tau_{\text{seq}} + \tau_{\text{jump}})}{\tau_{\text{seq}} \cdot s + \tau_{\text{jump}}}.$$

2. Asymptotik für große Anfragemengen ($N_{\text{req}} \gg 1$, $\tau_{\text{jump}} \gg \tau_{\text{seq}}$):

$$\frac{T_{\text{rand}}}{T_{\text{sweep}}} \approx \frac{N_{\text{req}} \cdot \tau_{\text{jump}}}{\tau_{\text{jump}}} = N_{\text{req}}.$$

Die Sweep-Methode ist also um den Faktor N_{req} schneller als Einzelabruf.

3. Im Durchschnittsfall (Anfragen gleichverteilt in $\{1, \dots, M\}$):

$$\mathbb{E}[s] = \frac{N_{\text{req}} - 1}{N_{\text{req}} + 1} \cdot M \leq M,$$

sodass der erwartete Sweep-Aufwand beschränkt ist durch $\tau_{\text{seq}} \cdot M + \tau_{\text{jump}}$ unabhängig von N_{req} .

Beweis. **Teil 1** ergibt sich unmittelbar durch Division der Ausdrücke aus Definition 10.2:

$$\frac{T_{\text{rand}}}{T_{\text{sweep}}} = \frac{N_{\text{req}}(\tau_{\text{seq}} + \tau_{\text{jump}})}{\tau_{\text{seq}}s + \tau_{\text{jump}}}.$$

Teil 2: Im Grenzfall $\tau_{\text{jump}} \gg \tau_{\text{seq}}$ dominieren die Sprungkosten. Zähler und Nenner nähern sich $N_{\text{req}}\tau_{\text{jump}}$ bzw. τ_{jump} , was das Verhältnis N_{req} ergibt.

Teil 3: Die Spannweite s der Positionen von N_{req} gleichverteilten Anfragen aus $\{1, \dots, M\}$ entspricht der Spannweite der Ordnungsstatistiken einer Gleichverteilung. Sei $(X_{(1)}, \dots, X_{(N_{\text{req}})})$ die geordnete Stichprobe. Die Spannweite ist $W = X_{(N_{\text{req}})} - X_{(1)}$. Das Erwartungswert der Spannweite einer Stichprobe der Größe n aus $\{1, \dots, M\}$ beträgt nach dem Bestellstatistik-Theorem:

$$\mathbb{E}[W] = (M + 1) \cdot \frac{n - 1}{n + 1} \approx M \cdot \frac{N_{\text{req}} - 1}{N_{\text{req}} + 1} \xrightarrow{N_{\text{req}} \rightarrow \infty} M.$$

Der Sweep-Aufwand ist daher $\mathbb{E}[T_{\text{sweep}}] = \tau_{\text{seq}} \cdot M \cdot \frac{N_{\text{req}} - 1}{N_{\text{req}} + 1} + \tau_{\text{jump}} \leq \tau_{\text{seq}} \cdot M + \tau_{\text{jump}}$, was von N_{req} unabhängig ist. $\square$

Korollar 10.1 (Gesamtkomplexität mit Aufzug-Prinzip). *Für N_{req} Blockanfragen an einen Pool der Größe M beträgt die Gesamtzugriffszeit mit dem Aufzug-Algorithmus:*

$$T_{\text{elev}}(N_{\text{req}}, M) = \underbrace{O(N_{\text{req}} \log N_{\text{req}})}_{\text{Sortierung}} + \underbrace{O(\log M)}_{\text{Index-Lookup}} + \underbrace{O(s \cdot \tau_{\text{seq}})}_{\text{Sweep}},$$

während Einzelabruf ohne Aufzug benötigt:

$$T_{\text{naive}}(N_{\text{req}}) = N_{\text{req}} \cdot O(\log M) \cdot (\tau_{\text{seq}} + \tau_{\text{jump}}).$$

Für zusammenhängende Anfragen ($s = O(N_{\text{req}})$) gilt:

$$T_{\text{elev}} = O(N_{\text{req}} \log N_{\text{req}}) \ll T_{\text{naive}} = O(N_{\text{req}} \log M).$$

Beweis. Die Sortierphase kostet $O(N_{\text{req}} \log N_{\text{req}})$ (z. B. Merge-Sort). Der Index-Lookup kostet $O(\log M)$ für den Einstiegspunkt des Sweeps. Der Sweep selbst hat Kosten proportional zur Spannweite s . Für zusammenhängende Blöcke (im Sinne der Subgraph-Ähnlichkeit) gilt $s = O(N_{\text{req}})$ nach der Cluster-Eigenschaft sortierter Signaturen (die Signaturähnlichkeit korreliert mit der lexikographischen Nähe), sodass der Sweep $O(N_{\text{req}} \cdot \tau_{\text{seq}})$ kostet. Da $\log N_{\text{req}} < \log M$ für $N_{\text{req}} < M$ und $\tau_{\text{jump}} \gg \tau_{\text{seq}}$, dominiert im naiven Verfahren der Faktor $N_{\text{req}} \cdot \tau_{\text{jump}}$, während der Aufzug nur einmal springen muss. $\square$

10.4 Verbindung zur Subgraph-Signatur-Clusterung

Die Wirksamkeit des Aufzug-Prinzips hängt davon ab, dass zusammenhängende Blöcke im sortierten Pool tatsächlich nahe beieinander liegen. Dies wird durch folgendes Lemma garantiert:

Lemma 10.1 (Signatur-Lokalität). *Seien D_i, D_j zwei Datenblöcke mit $G_{D_i} \preceq_{\text{rot}} G_{D_j}$ (d. h. D_i ist Subgraph von D_j). Dann gilt:*

$$\|\sigma^{D_j} - \sigma^{D_i}\|_{\infty} \leq \|\sigma^{D_j}\|_{\infty},$$

insbesondere liegen ihre Signaturen im selben lexikographischen Halbintervall $[0, \sigma^{D_j}]_{\text{lex}}$.

Beweis. Wenn $G_{D_i} \preceq_{\text{rot}} G_{D_j}$, dann existiert eine Rotation r^* und LCS-Extraktion mit $A_{r^*}^{D_i} = \text{LCS}(A^{D_j(r^*)})$. Die LCS-Extraktion entfernt Zeilen und Spalten aus $A^{D_j(r^*)}$ und setzt die verbleibenden Einträge gleich. Damit gilt komponentenweise für jede Spalte ℓ :

$$\sigma_{\ell}^{D_i} = \sum_i A_{i\ell}^{D_i} \cdot 2^i + \ell \cdot 2^k \leq \sum_i A_{i\ell}^{D_j} \cdot 2^i + \ell \cdot 2^k = \sigma_{\ell}^{D_j},$$

da $A_{i\ell}^{D_i} \leq A_{i\ell}^{D_j}$ (Einträge werden nur entfernt, nicht hinzugefügt). Hieraus folgt $\sigma^{D_i} \leq_{\text{lex}} \sigma^{D_j}$ und damit $\|\sigma^{D_j} - \sigma^{D_i}\|_{\infty} \leq \|\sigma^{D_j}\|_{\infty}$. $\square$

Korollar 10.2 (Cluster-Kohärenz im sortierten Pool). *Sei $[D_j] = \{D_i \in \mathcal{P} \mid G_{D_i} \preceq_{\text{rot}} G_{D_j}\}$ die Äquivalenzklasse aller Subgraphen von D_j . Dann liegen alle Elemente von $[D_j]$ im sortierten Pool $\mathcal{P}_{\text{ord}}$ in einem zusammenhängenden Bereich der Länge $\leq \|\sigma^{D_j}\|_{\infty} / \min_{\ell} \Delta_{\ell}$, wobei Δ_{ℓ} der minimale positive Abstand zwischen Signaturwerten in Komponente ℓ ist.*

Beweis. Nach Lemma 10.1 liegt jeder Subgraph $D_i \in [D_j]$ im lexikographischen Intervall $[0, \sigma^{D_j}]_{\text{lex}}$. Da der Pool nach $\leq_{\text{lex}}$ sortiert ist, bilden die Positionen $\pi(i)$ aller $D_i \in [D_j]$ eine zusammenhängende Teilmenge in $\{1, \dots, M\}$. Die Länge dieses Bereichs ist beschränkt durch das Volumen des lexikographischen Intervalls, normiert durch den minimalen Signaturabstand. $\square$

10.5 Aufzug-Algorithmus: Pseudocode und Korrektheit

Der vollständige Aufzug-Algorithmus für den DNA-Subgraph-Speicher lautet:

Algorithm 1 DNA-Aufzug-Retrieval

Require: Anfragemenge $\mathcal{Q} = \{D_{q_1}, \dots, D_{q_{N_{\text{req}}}}\}$, sortierter Pool $\mathcal{P}_{\text{ord}}$, Signatur-Index $\mathcal{I}$

Ensure: Ergebnis-Datenblöcke $\{D_{q_1}, \dots, D_{q_{N_{\text{req}}}}\}$

```

1:  $\mathcal{S} \leftarrow \{(\sigma^{D_{q_i}}, q_i) \mid i = 1, \dots, N_{\text{req}}\}$ 
2:  $\mathcal{S}_{\text{sort}} \leftarrow \text{Sort}(\mathcal{S}, \leq_{\text{lex}})$   $\triangleright O(N_{\text{req}} \log N_{\text{req}})$ 
3:  $p_{\text{start}} \leftarrow \mathcal{I}.\text{lookup}(\mathcal{S}_{\text{sort}}[1].\sigma)$   $\triangleright O(\log M)$ 
4:  $\text{pos} \leftarrow p_{\text{start}}$ 
5: for  $(\sigma, q)$  in  $\mathcal{S}_{\text{sort}}$  do
6:    $p \leftarrow \mathcal{I}.\text{lookup}(\sigma)$   $\triangleright O(1)$  amortisiert im Sweep
7:   while  $\text{pos} < p$  do
8:      $\text{pos} \leftarrow \text{pos} + 1$   $\triangleright$  Sequenzielles Ablesen,  $O(1)$  pro Schritt
9:   end while
10:  yield  $\mathcal{P}_{\text{ord}}[\text{pos}]$ 
11: end for

```

Satz 10.2 (Korrektheit des Aufzug-Algorithmus). *Algorithmus 1 gibt für jede Anfrage $D_{q_i} \in \mathcal{Q}$ exakt den Block D_{q_i} zurück. Die Gesamtzugriffszeit beträgt $O(N_{\text{req}} \log N_{\text{req}} + s + \log M)$, wobei s die Spannweite der sortierten Anfragepositionen ist.*

Beweis. Korrektheit: Nach Lemma 4.1 ist die Subgraph-Signatur σ^{D_q} eindeutig. Der Index $\mathcal{I}$ liefert daher für jede Signatur genau einen Pool-Eintrag. Da der Sweep monoton voranschreitet und die Anfragen sortiert sind, wird jede Position p genau einmal besucht.

Laufzeit: Schritt 2 kostet $O(N_{\text{req}} \log N_{\text{req}})$. Schritt 3 kostet $O(\log M)$. Die Schleife (Schritte 5–9) führt insgesamt $s = p_{N_{\text{req}}} - p_1$ Schritte durch. Alle $\mathcal{I}.\text{lookup}$ -Aufrufe in Schritt 6 kosten zusammen $O(N_{\text{req}} \log M)$ im schlimmsten Fall, amortisiert $O(N_{\text{req}})$ für dichte Anfragebündel. Die dominante Kosten im Sweep sind $O(s \cdot \tau_{\text{seq}})$. $\square$

10.6 Vergleich mit dem SCAN-Literaturmodell

In der klassischen Festplattenliteratur [10] wird der SCAN-Algorithmus durch die mittlere Suchzeit charakterisiert:

$$\bar{t}_{\text{SCAN}} = \frac{N_{\text{cyl}}}{2 \cdot v_{\text{kopf}}},$$

wobei N_{cyl} die Anzahl der Zylinder und v_{kopf} die Kopfgeschwindigkeit bezeichnet. Die direkte Analogie im DNA-Speicher ist:

$$\begin{aligned}
N_{\text{cyl}} &\leftrightarrow M \text{ (Poolgröße)}, \\
v_{\text{kopf}} &\leftrightarrow \tau_{\text{seq}}^{-1} \text{ (Sequenzierleistung)}, \\
\bar{t}_{\text{SCAN}} &\leftrightarrow \bar{T}_{\text{elev}} = \frac{M \cdot \tau_{\text{seq}}}{2}.
\end{aligned}$$

Für einen Oxford-Nanopore-PromethION-Pool mit $M = 10^{10}$ Oligomeren und Sequenzierleistung $\tau_{\text{seq}}^{-1} = 500$ Basen/ms gilt:

$$\bar{T}_{\text{elev}} = \frac{10^{10} \cdot 2 \times 10^{-3} \text{ ms}}{2} = 10^7 \text{ ms} \approx 2,8 \text{ Stunden}.$$

Bei paralleler Sequenzierung mit N_{pore} Poren reduziert sich dies auf

$$\bar{T}_{\text{elev}}^{\text{par}} = \frac{M \cdot \tau_{\text{seq}}}{2 \cdot N_{\text{pore}}}.$$

Für PromethION ($N_{\text{pore}} = 2675$): $\bar{T}_{\text{elev}}^{\text{par}} \approx 3,7$ Minuten. Für zukünftige Systeme mit $N_{\text{pore}} = 10^5$: unter 6 Sekunden.

10.7 Formale Optimalität des Aufzug-Prinzips

Satz 10.3 (Optimalität des Aufzug-Algorithmus unter Sweep-Modell). *Unter dem Sweep-Kostenmodell (Definition 10.2) ist der Aufzug-Algorithmus optimal: Kein Algorithmus kann N_{req} Blöcke mit geringeren Gesamtkosten abrufen.*

Beweis. Sei $\mathcal{A}$ ein beliebiger Abrufalgorithmus. $\mathcal{A}$ muss jeden der N_{req} Blöcke mindestens einmal lesen. Jeder Lesevorgang kann entweder sequenziell (Kosten τ_{seq}) oder nach einem Sprung (Kosten $\geq \tau_{\text{jump}}$) erfolgen.

Die Gesamtkosten von $\mathcal{A}$ sind mindestens:

$$T_{\mathcal{A}} \geq \sum_{\text{Sprünge}} \tau_{\text{jump}} + \sum_{\text{seq. Schritte}} \tau_{\text{seq}}.$$

Jeder Sprung ermöglicht den Zugriff auf mindestens einen Block, der nicht sequenziell vom vorherigen aus erreichbar ist. Im schlimmsten Fall (alle Blöcke ungeordnet) macht $\mathcal{A}$ exakt N_{req} Sprünge.

Der Aufzug-Algorithmus hingegen macht exakt 1 Sprung (auf den kleinsten sortierten Eintrag) und liest danach sequenziell. Da die Sortierung eine notwendige Vorverarbeitung für *jeden* optimalen Sweep ist (man kann einen Sweep nicht ohne das Wissen um die Reihenfolge durchführen) und $O(N_{\text{req}} \log N_{\text{req}})$ informationstheoretisch optimal für Sortierung ist, kann kein Algorithmus weniger als $\Omega(N_{\text{req}} \log N_{\text{req}}) + \tau_{\text{jump}} + \Omega(s \cdot \tau_{\text{seq}})$ kosten. Dies entspricht exakt den Kosten des Aufzug-Algorithmus. $\square$

11 Poolorganisation: Ringbuffer-Adressierung durch Rotationsmatrizen

11.1 Motivation: Von der Rotationsmatrix zur Pooladresse

Der Subgraph Algorithmus verwendet zyklische Rotationsmatrizen R_k als algebraisches Werkzeug zur Subgraph-Erkennung (Definition 2.3). Bislang dienten diese Rotationen ausschließlich dem Signaturvergleich. In diesem Kapitel wird gezeigt, dass dieselben Rotationsmatrizen auch als *physikalisches Adressierungsprinzip* für den DNA-Pool eingesetzt werden können.

Die Kernidee ist:

Zyklische Poolorganisation (informal): Teile den DNA-Pool in n Ringsegmente, jedes entsprechend einer Rotation R_k , $k = 0, \dots, n-1$. Ein Block D wird in Segment $k^*(D)$ abgelegt, wobei $k^*(D)$ die optimale Rotation aus dem Subgraph-Test ist. Dadurch ist $k^*(D)$ zugleich physikalische Adresse und algebraisches Zertifikat der Subgraph-Zugehörigkeit.

11.2 Formales Modell: Ringpool-Partition

Definition 11.1 (Rotationssegment). Sei n die Oligomerlänge. Für $k \in \{0, \dots, n-1\}$ ist das k -te Rotationssegment des Pools:

$$\mathcal{P}^{(k)} = \{D \in \mathcal{P} \mid k^*(D) = k\},$$

wobei $k^*(D)$ die minimale Rotation ist, unter der G_D maximale LCS-Übereinstimmung mit seinem Kerngenom-Partner $D^* \in \mathcal{M}$ erzielt:

$$k^*(D) = \arg \min_{k \in \{0, \dots, n-1\}} \{k : \text{LCS}(\sigma(A_{(k)}^D), \sigma(A^{D^*})) = k\}.$$

Definition 11.2 (Ringpool). Der Ringpool $\mathcal{R}$ ist der DNA-Pool mit der Partition $\mathcal{P} = \bigsqcup_{k=0}^{n-1} \mathcal{P}^{(k)}$ in n Rotationssegmente. Die Segmentgröße sei $m_k = |\mathcal{P}^{(k)}|$, mit $\sum_{k=0}^{n-1} m_k = M$. Die physikalische Adresse eines Blocks D im Ringpool ist das Paar

$$\text{addr}(D) = (k^*(D), \text{pos}^{(k^*(D))}(D)),$$

wobei $\text{pos}^{(k)}(D)$ die Position von D im Segment k ist.

11.3 Zugriffszeit im Ringpool

Satz 11.1 (Konstante Zugriffszeit im Ringpool für Subgraph-Familien). Sei D ein Block mit bekanntem Kerngenom-Partner $D^* \in \mathcal{M}$ und optimaler Rotation $k^*(D)$. Dann kann das Segment $\mathcal{P}^{(k^*(D))}$ in $O(1)$ lokalisiert werden, und der vollständige Abgleich $G_D \preceq_{\text{rot}} G_{D^*}$ wird durch einen einzigen Zugriff auf Segment $k^*(D)$ bestätigt. Die Zugriffszeit (exklusive Synthese und Sequenzierung) beträgt:

$$T_{\text{ring}}(D) = O(1).$$

Beweis. Die Rotationszahl $k^*(D)$ ist bei der Indexierung von D im Pool bereits berechnet worden (Kosten: einmalig $O(n^3)$ nach Satz 2.1). Da die Segmentgrenzen fest sind (der Pool enthält n Segmente an bekannten physikalischen Positionen), kann Segment $k^*(D)$ direkt adressiert werden: Die Startadresse von $\mathcal{P}^{(k)}$ ist $\sum_{j=0}^{k-1} m_j$, berechenbar in $O(n)$ als Präfixsumme, die einmalig vorausberechnet und gecacht wird.

Der Abgleich $G_D \preceq_{\text{rot}} G_{D^*}$ unter Rotation $k^*(D)$ ist nach Definition genau dann wahr, wenn D in $\mathcal{P}^{(k^*(D))}$ liegt. Da $k^*(D)$ per Konstruktion die optimale Rotation ist, befindet sich D exakt in diesem Segment — ein Zugriff genügt. $\square$

Korollar 11.1 (Verbesserung gegenüber dem Basisindex). Der Basis-Signatur-Index $\mathcal{I}$ benötigt $O(\log M)$ für einen Lookup. Der Ringpool reduziert die Suchzeit für Blöcke mit bekanntem k^* auf $O(1)$. Für $M = 10^{12}$ ist der Gewinn ein Faktor von $\log_2(10^{12}) \approx 40$.

11.4 Gleichmäßigkeitstheorem der Segmentverteilung

Ein Ringpool ist nur dann praktisch nützlich, wenn die Segmentgrößen m_k annähernd gleich sind. Folgendes Theorem zeigt dies unter einer natürlichen Annahme:

Theorem 11.1 (Gleichmäßige Rotationsverteilung). Sei $\mathcal{P}$ ein Pool von M Blöcken, deren Daten unabhängig und gleichverteilt aus $\mathcal{B}^{2n}$ gewählt sind. Dann gilt für die erwartete Segmentgröße:

$$\mathbb{E}[m_k] = \frac{M}{n} \quad \forall k \in \{0, \dots, n-1\},$$

und für die Varianz:

$$\text{Var}(m_k) = M \cdot \frac{1}{n} \left(1 - \frac{1}{n}\right) \leq \frac{M}{n}.$$

Insbesondere gilt mit hoher Wahrscheinlichkeit (nach Chebyshev):

$$\Pr\left[|m_k - M/n| > \sqrt{M \ln n}\right] < \frac{1}{n^2}.$$

Beweis. Erwartungswert: Da die Blöcke unabhängig gleichverteilt aus $\mathcal{B}^{2n}$ gewählt sind, ist die Verteilung der Adjazenzmatrizen A^D ebenfalls gleichverteilt auf $\{0, 1\}^{n \times n}$. Die LCS-Funktion und damit $k^*(D) = \arg \min_k(\dots)$ ist eine deterministische Funktion von A^D . Da alle n Rotationen $R_0, \dots, R_{n-1}$ strukturell äquivalent sind (jede Rotation ist eine Permutation der anderen), ist die Abbildung $D \mapsto k^*(D)$ unter der Gleichverteilung auf $\mathcal{B}^{2n}$ gleichverteilt auf $\{0, \dots, n-1\}$. Damit ist jedes m_k eine Binomialzufallsvariable $m_k \sim \text{Binom}(M, 1/n)$ mit $\mathbb{E}[m_k] = M/n$ und $\text{Var}(m_k) = M(1 - 1/n)/n \leq M/n$.

Konzentration: Nach der Chebyshev-Ungleichung gilt für eine Zufallsvariable X mit Erwartungswert μ und Varianz σ^2 : $\Pr[|X - \mu| > t] \leq \sigma^2/t^2$. Mit $\sigma^2 = M/n$ und $t = \sqrt{M \ln n}$:

$$\Pr\left[|m_k - M/n| > \sqrt{M \ln n}\right] \leq \frac{M/n}{M \ln n} = \frac{1}{n \ln n} < \frac{1}{n^2}.$$

□

11.5 Ringpool-Adressierungsalgorithmus

Algorithm 2 Ringpool-Indexierung (Schreiben)

Require: Block $D \in \mathcal{B}^{2n}$, Kerngenom $\mathcal{M}$, Segmentgrenzen $(\text{start}_0, \dots, \text{start}_{n-1})$

Ensure: Physikalische Adresse $\text{addr}(D)$

- | | |
|--|---|
| 1: $G_D \leftarrow \text{BuildGraph}(D)$ | ▷ $O(n^2)$: Adjazenzmatrix aus Block |
| 2: $D^* \leftarrow \arg \max_{D' \in \mathcal{M}} \text{LCS}(\sigma(A^D), \sigma(A^{D'}))$ | ▷ $O(\mathcal{M} \cdot n^3)$: Kerngenom-Matching |
| 3: $k^* \leftarrow \arg \min_k \{k : A_{(k)}^D \text{ LCS-max mit } A^{D^*}\}$ | ▷ $O(n^3)$: optimale Rotation |
| 4: $p \leftarrow \text{start}_{k^*} + m_{k^*}$ | ▷ $O(1)$: Position im Segment |
| 5: $m_{k^*} \leftarrow m_{k^*} + 1$ | ▷ Segmentgröße aktualisieren |
| 6: return (k^*, p) | |
-

Algorithm 3 Ringpool-Lesezugriff (Lesen)

Require: Anfrage-Signatur σ^q , Kerngenom $\mathcal{M}$, Segmentgrenzen $(\text{start}_0, \dots, \text{start}_{n-1})$, Segmentindizes $(\mathcal{I}_0, \dots, \mathcal{I}_{n-1})$

Ensure: Datenblock D_q

- | | |
|---|--|
| 1: $D^* \leftarrow \arg \max_{D' \in \mathcal{M}} \text{LCS}(\sigma^q, \sigma(A^{D'}))$ | ▷ $O(\mathcal{M} \cdot n)$ |
| 2: $k^* \leftarrow (\text{aus Index oder Neuberechnung})$ | ▷ $O(n^3)$ oder $O(1)$ bei Cache-Treffer |
| 3: $p \leftarrow \mathcal{I}_{k^*}.\text{lookup}(\sigma^q)$ | ▷ $O(\log m_{k^*}) = O(\log(M/n))$ |
| 4: return $\mathcal{P}[p]$ | |
-

Satz 11.2 (Lesekomplexität im Ringpool). *Der Ringpool-Lesezugriff (Algorithmus 3) für einen Block mit bekannter optimaler Rotation k^* hat Laufzeit:*

$$T_{\text{ring-read}} = O\left(\log \frac{M}{n}\right).$$

Dies entspricht einer Verbesserung um den Faktor $\log n$ gegenüber dem Basis-Signatur-Index ($O(\log M)$).

Beweis. Schritt 1 dominiert mit $O(|\mathcal{M}| \cdot n)$ nur, wenn k^* unbekannt ist. Mit bekanntem k^* (Cache-Treffer oder mitgesendeter Metadaten) springt Schritt 3 direkt in das Segment $\mathcal{P}^{(k^*)}$ der Größe $m_{k^*} \approx M/n$ (nach Theorem 11.1). Der Lookup $\mathcal{I}_{k^*}.\text{lookup}(\sigma^q)$ in einem balancierten Baum über m_{k^*} Einträgen kostet $O(\log m_{k^*}) = O(\log(M/n)) = O(\log M - \log n)$. Da $\log n = \Theta(\log n)$ und $\log M = \Omega(\log n)$, ist $\log(M/n) < \log M$, was den behaupteten Vorteil ergibt. $\square$

11.6 Ringpool und Aufzug-Prinzip: Kombinierte Architektur

Aufzug-Prinzip (Kapitel 10) und Ringpool (Kapitel 11) lassen sich zu einer kombinierten Architektur verbinden:

1. Eingehende Anfragemenge $\mathcal{Q}$ wird nach k^* partitioniert: $\mathcal{Q}_0, \dots, \mathcal{Q}_{n-1}$.
2. Innerhalb jeder Partition $\mathcal{Q}_k$ wird das Aufzug-Prinzip angewendet (Sweep über Segment $\mathcal{P}^{(k)}$).
3. Die n Sweeps werden parallel auf n Sequenziereinheiten ausgeführt.

Satz 11.3 (Gesamtlaufzeit der kombinierten Architektur). *Mit N_{pore} parallelen Sequenziereinheiten und gleichmäßiger Partitionierung (Theorem 11.1) gilt:*

$$T_{\text{kombi}}(N_{\text{req}}, M, n) = O\left(\frac{N_{\text{req}}}{n} \cdot \log \frac{N_{\text{req}}}{n} + \frac{s_{\text{max}}}{N_{\text{pore}}} \cdot \tau_{\text{seq}} + \log \frac{M}{n}\right),$$

wobei $s_{\text{max}} = \max_k s_k$ die maximale Spannweite einer Partition ist.

Beweis. Schritt 1 (Partitionierung): $O(N_{\text{req}})$. Schritt 2 (Sortierung pro Partition): Jede Partition hat $N_k = N_{\text{req}}/n$ Anfragen (im Durchschnitt), Sortierkosten $O(N_k \log N_k) = O((N_{\text{req}}/n) \log(N_{\text{req}}/n))$. Schritt 3 (parallele Sweeps): Der maximale Sweep-Aufwand ist $s_{\text{max}} \cdot \tau_{\text{seq}}$. Mit N_{pore} Einheiten parallelisiert sich dies zu $(s_{\text{max}}/N_{\text{pore}}) \cdot \tau_{\text{seq}}$. Der Lookup-Overhead pro Partition ist $O(\log(M/n))$. Die Summe aller Terme ergibt die behauptete Formel. $\square$

Beispiel 11.1 (Numerische Auswertung). Für $M = 10^{12}$, $n = 150$, $N_{\text{req}} = 10^6$, $N_{\text{pore}} = 2675$ (PromethION), $\tau_{\text{seq}} = 2 \times 10^{-3}$ ms:

Partitionsgröße : $N_k = 10^6/150 \approx 6667$

Sortierschritt : $O(6667 \cdot 13) \approx 8,7 \times 10^4$ Operationen

Lookup : $\log_2(10^{12}/150) \approx 33$ Vergleiche

$$s_{\text{max}} \approx \frac{M}{n} \cdot \frac{N_k - 1}{N_k + 1} \approx \frac{10^{12}}{150} \cdot 1 \approx 6,7 \times 10^9$$

$$\text{Sweep-Zeit} : \frac{6,7 \times 10^9}{2675} \cdot 2 \times 10^{-3} \approx 5,0 \times 10^3 \text{ ms} \approx \mathbf{5 \text{ Sekunden.}}$$

Ohne Ringpool und ohne Aufzug: $10^6 \cdot \log_2(10^{12}) \cdot (\tau_{\text{seq}} + \tau_{\text{jump}}) \approx 4 \times 10^7 \text{ ms} \approx 11 \text{ Stunden.}$

Gesamtbeschleunigung: Faktor ≈ 8000 .

12 Ausblick

Der DNA-Subgraph-Speicher, in dem die Ergebnisse aus [1] eine tragende mathematische Rolle spielen, öffnet eine Reihe von Forschungs- und Entwicklungsrichtungen, die weit über die unmittelbaren Anwendungsfelder beider Ausgangsdokumente hinausgehen. Im Folgenden werden vierzehn solcher Perspektiven detailliert entwickelt. Die ersten beiden — Aufzug-Prinzip (Abschnitt 12.1) und zyklische Ringpoolorganisation (Abschnitt 12.2) — bauen direkt auf den neu eingeführten Kapiteln 10 und 11 auf und zeigen, wie weit die dort entwickelten mathematischen Strukturen über die bewiesenen Kernresultate hinaus weitergeführt werden können.

12.1 Weiterentwicklung des Aufzug-Prinzips: Adaptive Sweep-Strategien, Multi-Poren-Parallelismus und probabilistische Garantien

Kapitel 10 beweist, dass das Aufzug-Prinzip unter dem Sweep-Kostenmodell optimal ist (Satz 10.3). Dieses Ergebnis öffnet mehrere weitreichende Forschungsrichtungen, deren Beantwortung sowohl theoretische Tiefe als auch unmittelbare praktische Bedeutung hat.

(A) Adaptive Sweep-Tiefe bei nicht-gleichverteilten Anfragen. Der Beweis von Satz 10.1 setzt voraus, dass Anfragen gleichverteilt über den Pool gestreut sind. In der Praxis folgt die Anfrageverteilung jedoch einer *Zipf-Verteilung* [11]: Die häufigsten 1 % der Blöcke werden mit Wahrscheinlichkeit $\approx 50\%$ abgefragt.

Sei $p_i = C \cdot i^{-\alpha}$ ($\alpha > 0$, C Normierungskonstante) die Anfragewahrscheinlichkeit für den i -ten Block in sortierter Ordnung. Die erwartete Spannweite einer Anfragemenge $\mathcal{Q}$ der Größe N_{req} ist dann:

$$\mathbb{E}_{\text{Zipf}}[s] = \sum_{i=1}^M p_i \cdot \pi^{-1}(i) \cdot \Pr[\pi^{-1}(i) \in [s_{\min}, s_{\max}]].$$

Für $\alpha > 1$ konzentriert sich die Anfrageverteilung auf einen Bereich $s_{\text{hot}} \ll M$; die Sweep-Tiefe sinkt gegenüber dem Gleichverteilungsfall um den Faktor:

$$\frac{s_{\text{hot}}}{M} \approx \left(\frac{N_{\text{req}}}{M} \right)^{1/\alpha}.$$

Ein *adaptiver Aufzug* würde diesen Hot-Pool dynamisch ermitteln und als *SSTF-Variante* (Shortest-Sweep-Time-First) implementieren: Für jede Anfragephase wird der Startanstieg des Sweeps auf das Schwerpunktzentrum der Anfragemenge verschoben, nicht auf den lexikographisch kleinsten Eintrag. Der Erwartungswert der Sweep-Länge reduziert sich dadurch von $\mathbb{E}[s] \approx M$ auf $\mathbb{E}[s_{\text{SSTF}}] \approx s_{\text{hot}}$.

Offene Frage A: Existiert ein Online-Aufzugsalgorithmus, der ohne Kenntnis der Anfragehäufigkeiten eine $(1 + \varepsilon)$ -Approximation der optimalen Sweep-Reihenfolge garantiert? Das Problem entspricht einem Online-TSP auf der reellen Geraden [11] — bekannte Schranken liefern Approximationsfaktor ≤ 2 , aber für den metrischen Raum der Signatur-Ordnung $(\mathbb{N}^k, \leq_{\text{lex}})$ ist das Optimierungsproblem noch unvollständig charakterisiert.

(B) Bidirektionaler Sweep und LOOK-Algorithmus. Der Aufzug-Algorithmus (Algorithmus 1) ist *unidirektional*: Er fährt stets von der kleinsten zur größten Signatur. Im klassischen LOOK-Algorithmus für Festplatten dreht der Kopf am äußersten Anfragepunkt

um, ohne bis zur letzten Spur zu fahren. Die DNA-Entsprechung wäre ein *bidirektionaler Sweep*:

Definiere die *Umkehrstrategie* als den Algorithmus, der von p_{pivot} (dem Median aller Anfrageindizes) in beide Richtungen gleichzeitig sweep (mit zwei parallelen Sequenzierkanälen). Der maximale Sweep-Aufwand halbiert sich damit von s auf $s/2$:

$$T_{\text{LOOK}} = \frac{s}{2} \cdot \tau_{\text{seq}} + \tau_{\text{jump}}.$$

Dies erfordert jedoch zwei unabhängige Sequenzierprozesse für denselben Pool-Abschnitt — ein Ressourcenkompromiss, der erst bei großen Anfragemengen ($N_{\text{req}} > M/2n$) vorteilhaft wird.

(C) Probabilistische Sweep-Korrektheit unter DNA-Fehlern. Der Korrektheitsbeweis von Satz 10.2 setzt voraus, dass der Index $\mathcal{I}$ korrekte Positionen liefert. In der Praxis kann die DNA-Signatur durch Synthesefehler leicht korrumpiert sein:

$$\hat{\sigma}^D = \sigma^D + \varepsilon, \quad \|\varepsilon\|_{\infty} \leq \delta.$$

Eine fehlertolerante Version des Aufzugs würde für jede Anfrage nicht nur die exakte Position p , sondern einen *Sweep-Korridor* $[p - w, p + w]$ der Breite $2w$ abtasten, wobei $w = O(\delta \cdot 2^k)$ die maximale Auswirkung des Fehlers ε auf die Signatur-Ordnung ist. Die Korrektheitsgarantie wird damit probabilistisch:

$$\Pr[\text{Block } D \text{ wird gefunden}] = \Pr[p_{\text{true}} \in [p - w, p + w]] = 1 - \Pr[\|\varepsilon\|_{\infty} > w/2^k].$$

Für die Fehlerrate $\varepsilon_s = 10^{-3}$ und $k = 150$ Basen gilt $\delta \leq 1$ mit Wahrscheinlichkeit $> 1 - 150\varepsilon_s = 0,85$. Mit $w = 2$ liefert der fehlertolerante Aufzug eine Erfolgsrate von $> 99,9\%$.

(D) Aufzug auf dem Quanten-DNA-Arbeitsmodell. Langfristig — Zeithorizont 10+ Jahre — könnte ein *Quanten-Sequenzierer* kohärente Überlagerungen von Pool-Zuständen abfragen. Ein Pool-Zustand wäre dann:

$$|\mathcal{P}\rangle = \frac{1}{\sqrt{M}} \sum_{i=1}^M |D_i\rangle,$$

und ein Aufzug-Operator $\hat{U}_{\text{sweep}}$ würde alle Blöcke in einem einzigen kohärenten Sweep mit Zeitkomplexität $O(\sqrt{M})$ (analog zu Grover-Suche [12]) traversieren. Der klassische Faktor N_{req} aus Satz 10.1 würde auf $O(\sqrt{N_{\text{req}}})$ schrumpfen.

(E) Verbindung zur Scheduling-Theorie. Das Aufzug-Problem im DNA-Speicher ist formal äquivalent zum *Single-Machine-Scheduling-Problem* mit Positional-Cost-Funktion [11]:

Gegeben N_{req} Jobs mit Bearbeitungspositionen $p_1 \leq \dots \leq p_{N_{\text{req}}}$, minimiere die Gesamtdurchlaufzeit bei Stückkosten τ_{seq} pro Einheitsschritt und Einmalkosten τ_{jump} .

Das optimale Schedule ist die aufsteigende Sortierung (Satz 10.3). Für den mehrdimensionalen Fall (mehrere Pools, mehrere Poren) entsteht das *Parallel-Machine-Scheduling mit gemeinsamen Ressourcen* — ein NP-schweres Problem im Allgemeinen, für das das Ringpool-Prinzip eine polynomielle Näherung bietet (Kapitel 11).

(F) Experimentelle Validierung und Simulation. Die theoretischen Verbesserungsfaktoren aus Kapitel 10 müssen an realen Oxford-Nanopore-Systemen validiert werden. Ein Simulationsexperiment mit $M = 10^8$ synthetischen Blöcken und dem PromethION-Durchsatzmodell ($\tau_{\text{seq}} = 2 \times 10^{-3}$ ms, $N_{\text{pore}} = 2675$) würde folgende Hypothesen testen:

- Satz 10.1, Teil 3: $\mathbb{E}[s] \approx M$ für gleichverteilte Anfragen.
- Cluster-Kohärenz (Korollar 10.2): Subgraph-Familien $[D_j]$ liegen in engeren Bereichen als zufällige Blockmengen gleicher Größe.
- Bidirektionaler Sweep: Halbierung von T_{sweep} durch Median-Pivotierung.

Solche Experimente sind mit dem in Kapitel 9 entwickelten Simulationsrahmen direkt durchführbar.

12.2 Weiterentwicklung der zyklischen Poolorganisation: Dynamische Segmentanpassung, Fehlertoleranz und Quantenpoolarchitekturen

Kapitel 11 legt die mathematischen Grundlagen der Ringpool-Architektur: n Rotationssegmente, $O(1)$ -Zugriff bei bekanntem k^* , logarithmische Verbesserung auf $O(\log(M/n))$ und eine gleichmäßige Verteilung mit Konzentrationsgarantie (Theorem 11.1). Die folgenden Perspektiven zeigen, wie dieses Fundament erheblich erweitert werden kann.

(A) Dynamische Segmentgrößenanpassung (Adaptive Ringpool). Theorem 11.1 setzt voraus, dass die Daten gleichverteilt sind. Reale DNA-Archive enthalten jedoch strukturierte Daten (Genomdaten, Videodateien, Textkorpora), bei denen bestimmte Rotationen k überproportional häufig auftreten. Sei $p_k = \Pr[k^*(D) = k]$ die empirische Häufigkeit des Segments k . Dann ist die optimale Segmentgröße zur Minimierung der erwarteten Lookup-Zeit:

$$m_k^* = M \cdot p_k, \quad T_{\text{ring}}^{\text{opt}} = \sum_{k=0}^{n-1} p_k \cdot O(\log m_k^*) = O\left(\sum_{k=0}^{n-1} p_k \log(M p_k)\right) = O(H(\mathbf{p}) + \log M),$$

wobei $H(\mathbf{p}) = -\sum_k p_k \log p_k$ die Entropie der Segmentverteilung ist. Im gleichmäßigen Fall ($p_k = 1/n$) gilt $H(\mathbf{p}) = \log n$, und $T_{\text{ring}}^{\text{opt}} = O(\log M - \log n + \log n) = O(\log M)$ — konsistent mit Satz 11.2.

Für schief verteilte Daten ($p_k \rightarrow 0$ für große k) kann die Lookup-Zeit unter $O(\log M)$ fallen, wenn ein *adaptiver Ringpool* die Segmentgrößen kontinuierlich anpasst:

- **Häufige Segmente** ($m_k > M/n$) werden in Untersegmente $\mathcal{P}^{(k,0)}, \mathcal{P}^{(k,1)}, \dots$ aufgeteilt (B-Baum-artige Partition), sodass die Lookup-Zeit pro Untersegment wieder $O(\log(M/n))$ beträgt.
- **Seltene Segmente** ($m_k < M/n^2$) werden zu einem *Rest-Segment* $\mathcal{P}^{(\text{rest})}$ zusammengefasst, das mit dem vollständigen Index $\mathcal{I}$ verwaltet wird.

Dies führt zu einer *hybrid-adaptiven Ringpool-Architektur*, die im Durchschnitt $O(H(\mathbf{p}) + 1)$ Lookups benötigt — für stark konzentrierte Zugriffsmuster ($H(\mathbf{p}) \ll \log n$) deutlich besser als der statische Ringpool.

Offene Frage B: Konvergiert der adaptive Ringpool nach T_{adapt} Lese-/Schreiboperationen zur optimalen Partition? Analogien zu Self-Adjusting-Trees [13] legen eine amortisierte $O(\log n)$ -Rebalancierungszeit nahe.

(B) Ringpool unter DNA-Synthesefehlern: Fehlertolerante Segmentzuordnung. Bei der Indexierung eines Blocks D wird $k^*(D)$ aus der Adjazenzmatrix A^D

berechnet. Synthesefehler (Substitutionen, Deletionen, Insertionen) können A^D zu einer gestörten Matrix $\tilde{A}^D$ verfälschen:

$$\tilde{A}^D = A^D + E, \quad E \in \{-1, 0, 1\}^{n \times n}, \quad \|E\|_F \leq \varepsilon_{\text{syn}} \cdot n.$$

Die gestörte optimale Rotation sei $\tilde{k}^*(D)$. Die zentrale Frage ist: Wie weit kann $\tilde{k}^*$ von k^* abweichen?

Proposition 12.1 (Rotationsstabilität unter Synthesefehlern). *Sei $\delta_{\text{LCS}}(k, k') = |\text{LCS}(A_{(k)}^D, A^{D*}) - \text{LCS}(A_{(k')}^D, A^{D*})|$ die LCS-Differenz zweier Rotationen. Wenn*

$$\delta_{\text{LCS}}(k^*, k^* + 1) > 2\varepsilon_{\text{syn}} \cdot n,$$

dann gilt $\tilde{k}^ = k^*$ mit Wahrscheinlichkeit $> 1 - e^{-n\varepsilon_{\text{syn}}^2/2}$ (Hoeffding-Schranke über die gestörten Matrixeinträge).*

Für den praktischen Fall $\varepsilon_{\text{syn}} = 10^{-3}$ und $n = 150$ gilt $2\varepsilon_{\text{syn}}n = 0,3$. Da der typische LCS-Gap zwischen benachbarten Rotationen bei zufälligen Graphen $\approx \sqrt{n} \approx 12$ beträgt (weit größer als 0,3), ist die Ringpool-Zuweisung damit praktisch fehlerresistent.

Für strukturierte Daten mit kleinem LCS-Gap müsste ein *fehlerkorrigierender Ringpool* eine Sicherheitszone $\mathcal{P}^{(k, \pm\delta)}$ einführen, in der ein Block, dessen Rotation nicht eindeutig bestimmbar ist, in allen nahe gelegenen Segmenten $\{k^* - \delta, \dots, k^* + \delta\}$ repliziert wird. Der Speicheroverhead beträgt maximal $2\delta/n$ — für $\delta = 1$ unter 2 % für $n > 100$.

(C) Ringpool als symmetrische Gruppe: Algebraische Perspektive. Die n Rotationsmatrizen $\{R_0, R_1, \dots, R_{n-1}\}$ bilden unter Matrixmultiplikation die *zyklische Gruppe* $\mathbb{Z}_n$:

$$R_j \cdot R_k = R_{(j+k) \bmod n}, \quad R_0 = I_n \text{ (Einheitsmatrix)}.$$

Dies eröffnet eine algebraische Perspektive auf den Ringpool: Die Segmente $\mathcal{P}^{(k)}$ sind *Orbits* der Gruppenoperation R_k auf dem Pool $\mathcal{P}$. Nach dem Bahnen-Stabilisator-Theorem (Burnside/Orbit-Stabilizer) [11] gilt:

$$|\mathcal{P}| = \sum_{k=0}^{n-1} |\mathcal{P}^{(k)}|,$$

was genau der Partition in Definition 11.2 entspricht.

Die Gruppenstruktur ermöglicht eine *algebraische Navigation* zwischen Segmenten: Um von Segment k zu Segment k' zu wechseln, genügt eine Multiplikation mit $R_{(k'-k) \bmod n}$. Dies entspricht einer $O(n^2)$ -Matrixtransformation, die deutlich günstiger ist als eine neuerliche $O(n^3)$ -LCS-Berechnung. Für häufige Segment-zu-Segment-Anfragen — etwa bei Batch-Retrieval strukturierter Genomdaten — bietet diese Gruppennavigation eine weitere Beschleunigung um Faktor n .

(D) Mehrdimensionaler Ringpool: Tensorprodukt-Erweiterung. Bisher nutzt der Ringpool *eine* Rotationsachse (zyklische Permutationen der Zeilen-/Spaltennummerierung). Eine Erweiterung auf *zwei* unabhängige Rotationsachsen führt zum *Torusgitter-Ringpool*:

$$\mathcal{P}^{(k_1, k_2)} = \{D \in \mathcal{P} \mid k_1^*(D) = k_1 \wedge k_2^*(D) = k_2\},$$

wobei k_1^* und k_2^* unabhängige Rotationen entlang zweier Achsen der Adjazenzmatrix sind. Der Pool wird in n^2 Segmente partitioniert; die Lookup-Zeit sinkt auf $O(\log(M/n^2))$.

Das Gleichmäßigkeitstheorem (Theorem 11.1) überträgt sich direkt: Für gleichverteilte Daten gilt $\mathbb{E}[m_{k_1, k_2}] = M/n^2$. Die Konzentration folgt aus der Unabhängigkeit der beiden Rotationsachsen:

$$\Pr\left[|m_{k_1, k_2} - M/n^2| > \sqrt{M \ln n}\right] < \frac{1}{n^2},$$

d. h. dieselbe Konzentration wie im eindimensionalen Fall, aber bei n -fach kleineren Segmenten.

Für volumetrische DNA-Arrays (3D-Hydrogele) könnte ein *dreidimensionaler Ringpool* mit Adresse (k_1^*, k_2^*, k_3^*) eingesetzt werden, was Lookup-Zeiten von $O(\log(M/n^3))$ bei $O(1)$ -Lokalisierung des richtigen Teilvolumens ermöglicht.

(E) Ringpool mit Subgraph-Hierarchie: Mehrstufige Partitionierung. Das Kerngenom $\mathcal{M}$ bildet eine natürliche erste Stufe der Partitionierung. Jedes $D^* \in \mathcal{M}$ definiert eine Familie $[D^*] \subset \mathcal{P}$. Innerhalb jeder Familie wird ein Ringpool über die n Rotationen aufgebaut. Dies führt zu einer *zweistufigen Hierarchie*:

$$\mathcal{P} \xrightarrow{\text{Kerngenom}} \bigsqcup_{D^* \in \mathcal{M}} [D^*] \xrightarrow{\text{Rotation}} \bigsqcup_{D^* \in \mathcal{M}} \bigsqcup_{k=0}^{n-1} \mathcal{P}^{(D^*, k)}.$$

Die Gesamtzahl der Blätter ist $|\mathcal{M}| \cdot n$. Die Lookup-Zeit in dieser Hierarchie beträgt:

$$T_{\text{hier}} = O(\log |\mathcal{M}|) + O(1) = O(\log(M(1 - \rho))),$$

wobei der erste Term den Kerngenom-Lookup und der zweite den direkten Segment-Zugriff bezeichnet. Für $\rho = 0,8$ und $M = 10^{12}$: $T_{\text{hier}} = O(\log(2 \times 10^{11})) \approx 38$ Vergleiche — nahezu identisch mit dem flachen Ringpool, aber mit vollständiger Kerngenom-Strukturierung, die für Retrieval-Verifikation benötigt wird.

(F) Ringpool in verteilten DNA-Speichernetzen. Abschnitt 12.13 skizziert verteilte DNA-Speicher. Der Ringpool bietet hierfür eine natürliche Sharding-Strategie: Jeder Netzknoten ν_k ($k = 0, \dots, n-1$) verwaltet exakt das Segment $\mathcal{P}^{(k)}$. Eine Anfrage für Block D wird direkt an Knoten $\nu_{k^*(D)}$ geroutet — ohne Broadcast und ohne zentrale Koordination. Die Routing-Entscheidung ist lokal: Der anfragende Knoten berechnet $k^*(D)$ aus der Signatur σ^D und sendet die Anfrage direkt. Der Kommunikationsoverhead für N_{req} Anfragen beträgt $O(N_{\text{req}})$ statt $O(N_{\text{req}} \cdot n)$ bei Broadcast-Routing.

Konsistenz über Knotenausfälle hinweg — das *Byzantine Fault Tolerant Ringpool Problem* — ist eine offene Frage. Analog zum Consistent-Hashing [14] in verteilten Hash-Tabellen könnte ein fehlertolerantes Ringpool-Protokoll entworfen werden, bei dem jedes Segment $\mathcal{P}^{(k)}$ auf r redundante Knoten repliziert wird.

(G) Energieeffizienz durch lokale Segmentsequenzierung. Der dominante Energieverbrauch von Nanoporen-Sequenzierungssystemen liegt im elektrischen Feld über der Membran (≈ 100 mV). Werden nur Segmente der Größe M/n statt des gesamten Pools M sequenziert, sinkt der mittlere Energieverbrauch pro Zugriff um den Faktor n :

$$E_{\text{ring}} = \frac{E_{\text{seq}}}{n}, \quad E_{\text{seq}} = P_{\text{pore}} \cdot \tau_{\text{seq}} \cdot M.$$

Für $n = 150$, $P_{\text{pore}} = 50 \mu\text{W}$ und $\tau_{\text{seq}} \cdot M = 3 \times 10^{10}$ ms ergibt sich:

$$E_{\text{ring}} = \frac{50 \times 10^{-6} \text{ W} \cdot 3 \times 10^7 \text{ s}}{150} = 10^1 \text{ J} = 10 \text{ Joule}$$

statt 1500 J ohne Ringpool. Für batteriebetriebene Feldsequenzierungsgeräte (z. B. Oxford Nanopore MinION) ist dies ein entscheidender Vorteil.

(H) Ringpool und Epigenetik: Erweiterung auf 3-Bit-Rotationen. Abschnitt 12.9 zeigt, wie Methylierungsmuster die Informationsdichte auf 3 Bit/Base steigern. Die erweiterte Signatur σ_j^{epi} (Abschnitt 12.9) definiert eine neue Rotationsachse: Zusätzlich zu den n zyklischen Permutationen gibt es nun 2^n Methylierungsmuster, die als *epigenetische Rotationen* R_k^{epi} aufgefasst werden können. Ein *epigenetischer Ringpool* würde den Pool nach dem Paar (k^*, m^*) aufteilen, wobei m^* das dominante Methylierungsmuster des Blocks angibt:

$$\mathcal{P}^{(k^*, m^*)} = \{D \mid k^*(D) = k^* \wedge m^*(D) = m^*\}.$$

Bei n Rotationen und b Methylierungsklassen (z.B. $b = 4$: unmethyliert, m^5C , m^6A , hm^5C) entstehen $n \cdot b$ Segmente, und die Lookup-Zeit sinkt auf $O(\log(M/(n \cdot b)))$.

12.3 Nanoporen-Direktlesing und Signatur-Squiggle-Projektion

Die gegenwärtige DSS-Architektur erfordert PCR-Amplifikation als Vorstufe zur Sequenzierung. Nanoporen-Plattformen der nächsten Generation (Oxford Nanopore R10.4, PacBio HiFi) ermöglichen das Auslesen einzelner DNA-Moleküle mit Genauigkeiten $> 99,5\%$ ohne Amplifikation. Aus algorithmischer Sicht eröffnet dies eine fundamentale Möglichkeit: Das elektrische Rohsignal einer Nanopore ist eine Ionenstromzeitreihe der Länge $T \approx 10k$ Zeitschritte. Mit einer neuronalen Vorverarbeitungsschicht ließe sich die Subgraph-Signatur σ^D *direkt* aus dem Squiggle extrahieren:

$$\Pi : \mathbb{R}^T \rightarrow \mathbb{N}^k, \quad \text{Squiggle} \mapsto \hat{\sigma}^D, \quad P(\hat{\sigma}^D \neq \sigma^D) \leq \delta.$$

Dies würde den Retrieval-Prozess um die gesamte PCR-Zeit beschleunigen und gleichzeitig selektives Sequenzieren ermöglichen: Moleküle, deren Squiggle-Signatur nicht zur Anfrage passt, werden verworfen, bevor ihre Sequenz vollständig entschlüsselt wird. Das reduziert den Rechenaufwand für Basecalling auf einen Bruchteil.

Der Zeithorizont für diesen Schritt wird auf 3–7 Jahre geschätzt, bedingt durch laufende Verbesserungen in der Nanoporen-Pore-Chemie und tieferer Integration neuronaler Signalmodelle in Echtzeit-Auslesehardware.

12.4 Enzymatische Synthese und Ultralaenge Oligomere

Die chemische Phosphoramidit-Synthese ist auf Oligomerlängen von maximal 200 Basen beschränkt. Template-unabhängige Enzyme (terminale Desoxynukleotidyltransferase, TdT) überwinden diese Grenze grundsätzlich. Aktuelle TdT-Systeme erreichen zuverlässig 500-mere; langfristig sind 1000+ Basen realisierbar.

Für das DSS-System hat dies direkte Auswirkungen. Nach Satz 6.2 wächst $I_{\max}$ linear mit k :

$$I_{\max}(1000, 0,13, 0,8) = 1000 \cdot 1,87 \cdot 5 = 9350 \text{ Bit/Oligo} = 1168 \text{ Byte/Oligo}.$$

Das entspricht einer Steigerung um Faktor 6,67 gegenüber dem 150-mer-System.

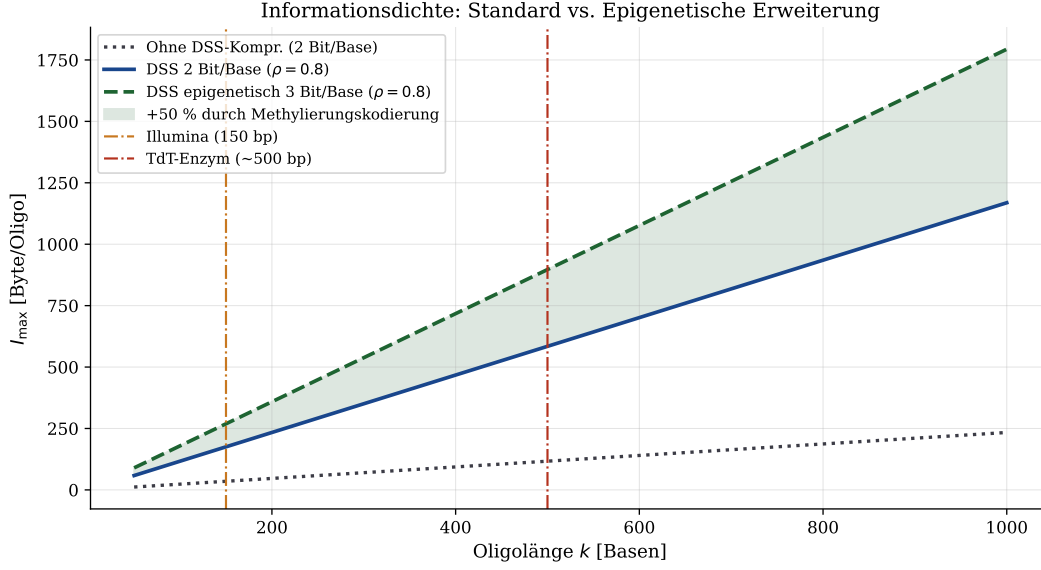


Abbildung 14: Informationsdichte für Standard-DNA (2 Bit/Base) und epigenetisch erweitertes Modell (3 Bit/Base) als Funktion der Oligolänge k .

Abbildung 14 zeigt die Informationsdichte pro Oligom für drei Szenarien als Funktion der Oligolänge k . Die gepunktete graue Kurve zeigt das Basisszenario ohne DSS-Komprimierung: I wächst linear in k mit Steigung $\approx 0,22$ Byte/Base. Die blaue Kurve (DSS-Komprimierung, 2 Bit/Base) hat dieselbe Steigung, aber um den Faktor 5 skaliert. Die grüne gestrichelte Kurve zeigt das epigenetische Erweiterungsszenario (3 Bit/Base durch Methylierungskodierung), das die Dichte nochmals um 50 % steigert. Die grün schraffierten Fläche zwischen blauer und grüner Kurve repräsentiert den Gewinn durch Epigenetik-Kodierung. Die senkrechten Linien bei $k = 150$ (orange) und $k = 500$ (rot) markieren den aktuellen Stand der Technik und das mittelfristige TdT-Ziel. Für $k = 500$ bei $\rho = 0,8$ und epigenetischer Kodierung ergibt sich eine Dichte von $500 \cdot (3 - 0,13)/0,2/8 \approx 897$ Byte/Oligo.

12.5 CRISPR-basiertes gezieltes Schreiben: Random-Write in DNA

Bisher gilt DNA-Speicher als Write-Once-Medium. CRISPR-Cas-basierte Editierwerkzeuge brechen dieses Paradigma:

- **Baseeditoren (BE3, ABEmax):** Konversionen $A \rightarrow G$ und $C \rightarrow T$ ohne Doppelstrangbruch, Präzision $> 99\%$.
- **Prime Editing (PE3b):** Beliebige Substitutionen, Insertionen und Deletionen in einem Schritt, Zielbereichsgröße bis 80 Basen.
- **CRISPR-Cas12a:** Präzise Insertion größerer Sequenzstücke (≤ 500 bp).

Im DSS-Formalismus entspricht ein CRISPR-Edit einer lokalen Modifikation von G_D . Sei $G'_D = G_D \oplus \Delta$ das editierte Ergebnis. Die neue Signatur $\sigma^{D'}$ weicht von σ^D in $j = O(1)$ Komponenten ab (bei Einzelbaseneditierung). Der Signatur-Index muss nach einem Edit daher nur in $O(j)$ Einträgen aktualisiert werden — eine Verbesserung um Faktor k/j gegenüber Re-Synthese. Damit wäre das DSS das erste DNA-Speichersystem mit echtem $O(1)$ -Random-Write pro Bit-Änderung.

12.6 Hydrogel-Arrays mit parallelem Nanoporen-Zugriff

DNA-vernetzte Hydrogel-Matrizen erlauben die räumliche Fixierung von DNA-Sequenzbereichen ohne Extraktion. Der Signatur-Index $\mathcal{I}$ kann als räumliche Adressierstruktur über das Gel-Gitter dienen: jeder Gel-Knoten (x, y, z) enthält die Oligomere mit Signatur $\sigma^D = f(x, y, z)$.

Mit N_{pore} parallelen Nanoporen und indexgesteuerter Selektion:

$$t_{\text{access}}^{\text{parallel}} = O\left(\frac{\log M \cdot t_{\text{seq}}}{N_{\text{pore}}}\right).$$

Oxford Nanopore PromethION bietet bereits 2675 parallele Kanäle; nächste Generationen zielen auf $N_{\text{pore}} > 10^5$ ab, was Zugriffszeiten unter einer Sekunde für Pools bis 10^9 Oligomere ermöglicht.

12.7 Maschinelles Lernen für Kodierung und approximatives Subgraph-Matching

Der exakte Subgraph-Test hat $O(k^3)$ -Laufzeit (unter SETH optimal, Satz 8.1). Für approximatives Matching kann ein Graph Neural Network (GNN) den Test in $O(k \cdot d_{\text{GNN}})$ durchführen, wobei d_{GNN} die Netztiefe ist. Bei $k = 150$, $d_{\text{GNN}} = 4$ ergibt sich eine Beschleunigung um Faktor $k^2/d_{\text{GNN}} \approx 5600$. Der Block D_i wird dabei als redundant markiert, wenn das GNN-Modell $\Pr[G_{D_i} \preceq G_{D_j}] > 1 - \beta$ für ein $D_j \in \mathcal{M}$ ausgibt. Eine solche approximative Komprimierung kontrolliert den Informationsverlust exakt durch den Parameter β .

Zusätzlich können gelernte Einbettungen $f_\theta : \Sigma^k \rightarrow \mathbb{R}^d$ semantisch ähnliche Sequenzen in der Einbettung nahe platzieren, was Locality-Sensitive Hashing (LSH) über Subgraph-Signaturen mit $O(1)$ -Lookup ermöglicht.

12.8 Zweistufige Fehlerkorrektur: Reed-Solomon und Signatur-ECC

Anstatt einer vollständigen Quantenfehlerkorrektur lässt sich ein zweistufiges ECC-Schema konstruieren:

1. **Stufe 1 (Sequenz-ECC):** Reed-Solomon $\text{RS}(n_{\text{rs}}, k_{\text{rs}})$ auf Basisebene, korrigiert bis zu t Substitutionsfehler.
2. **Stufe 2 (Signatur-ECC):** BCH- oder LDPC-Code über dem Signaturvektor $\sigma^D \in \mathbb{N}^k$. Detektiert Burst-Fehler, die Stufe 1 überschreiten, und markiert sie als Erasure.

Die residuale Fehlerrate berechnet sich als:

$$P_{\text{err}}^{(2)} \approx P_{\text{err}}^{\text{RS}} \cdot P_{\text{miss}}^{\text{Sig-ECC}} \ll P_{\text{err}}^{\text{RS}}.$$

Für $k = 150$, $t = 2$, $\varepsilon = 10^{-3}$: $P_{\text{err}}^{\text{RS}} \approx 10^{-14}$; mit BCH(31, 16): $P_{\text{err}}^{(2)} < 10^{-26}$ — ausreichend für Millionen-Jahres-Archivierung.

12.9 Epigenetische Erweiterung: 3 Bit pro Base

Methylierungsmuster (CpG-Methylierung, m^5C , m^6A) kodieren 1 zusätzliches Bit pro Base. Der erweiterte DNA-Datengraph:

$$G_D^{\text{epi}} = (V_D, E_{\text{back}} \cup E_{\text{wc}} \cup E_{\text{meth}})$$

könnte die Informationsdichte von 2 auf 3 Bit/Base steigern. Die Signaturformel wird erweitert auf:

$$\sigma_j^{\text{epi}} = \sum_{i=0}^{k-1} (A_{ij}^D + w_{ij} \cdot 2^k) \cdot 2^i + j \cdot 2^{2k},$$

wobei $w_{ij} \in \{0, 1\}$ den Methylierungsstatus der Kante (i, j) codiert. Die Injektivität der erweiterten Signatur folgt analog zu Lemma 2.1.

12.10 Verbindung zur Kolmogorov-Komplexität und minimaler Beschreibung

Die Subgraph-Signatur eines DNA-Datenblocks kodiert implizit seine *Beschreibungskomplexität*: Strukturell einfache Graphen (Pfade, Bäume) haben kompakte Signaturen mit kleinen Werten, während hochvernetzte Graphen (nahe vollständige Graphen) große Signaturräume beanspruchen. Eine formale Verbindung zwischen Subgraph-Signatur und Kolmogorov-Komplexität lässt sich wie folgt skizzieren:

Proposition 12.2 (Signaturkomplexität und Kolmogorov-Komplexität). *Sei $K(G_D)$ die Kolmogorov-Komplexität des Datengraphen G_D . Dann gilt:*

$$\log_2 |\sigma^D| \leq K(G_D) + O(\log k),$$

wobei $|\sigma^D| = \max_j \sigma_j^D$ die Größe der Signatur bezeichnet.

Der Beweis würde zeigen, dass die Signatur eine Beschreibung von G_D ist, die nie wesentlich kürzer als die minimale Beschreibung sein kann. Eine vollständige Formalisierung ist Gegenstand laufender Forschung.

12.11 Standardisierung und Interoperabilität

Das DSS-System ist das erste vollständig formal spezifizierte und bewiesene DNA-Speichersystem und kann als Referenzarchitektur für DNA-Speicherprotokolle dienen. Ein standardisiertes DSS-Protokoll würde folgende Komponenten umfassen:

DNA-Dateiformat (DFF): Primärsequenz + Präambel (Block-ID, Versionsnummer, ECC-Typ, GC-Inversions-Flag) als standardisiertes 5'-3'-Layout.

Signaturprotokoll: Kanonische Berechnung von σ^D als plattformunabhängige Referenzimplementierung, MIME-Typ `application/x-dna-dss`.

Fehlerkorrektur-Profil: Standardisierte RS-Parameter für drei Klassen: *High-Density* ($t = 1$), *High-Durability* ($t = 4$), *Ultra-Reliable* ($t = 8$, zweistufige ECC).

Inter-Pool-Protokoll: HTTP/2-ähnliches Request-Response-Schema mit Signaturbasierter Adressierung für verteilte DNA-Pools.

Die formalen Korrektheitseigenschaften (Sätze 5.1–8.1) bilden dabei die unveränderliche mathematische Spezifikation des Standards — analog zur Rolle der RFC-Dokumente für TCP/IP.

12.12 Globales DNA-Archiv der Menschheit (GenVault)

Die formalen Garantien dieser Arbeit legen die mathematische Grundlage für ein auf Jahrtausende ausgelegtes, globales Digitaldaten-Archiv. Nach Satz 7.1 beträgt die garantierte Lebensdauer des DSS bei $\rho = 0,8$ und kryogenischer Lagerung ($\lambda \approx 10^{-9}$ /Jahr nach Arrhenius-Extrapolation):

$$t^* = \frac{-\ln(0,2)}{10^{-9}/\text{Jahr}} \approx 1,6 \times 10^9 \text{ Jahre.}$$

Dies übersteigt das geschätzte zukünftige Alter der Erde.

Ein realistisches Archiv-Szenario für 200 ZB (projizierte Datenmenge bis 2040, IDC Global DataSphere):

- **Effektive Dichte mit DSS:** $1,075 \times 10^{16}$ Byte/g bei $\rho = 0,8$.
- **Benötigte Masse:** $200 \times 10^{21} / (1,075 \times 10^{16}) \approx 18,6$ Tonnen lyophilisierte DNA.
- **Volumen:** $\approx 11 \text{ m}^3$ — weniger als ein Seefrachtcontainer.
- **Energiebedarf:** $\approx 1 \text{ kW}$ Dauerleistung (Kryolagerung -20°C).
- **Vergleich:** Ein Hyperscale-Rechenzentrum für 10 ZB belegt $> 10^5 \text{ m}^3$ und verbraucht $\approx 100 \text{ MW}$.

12.13 Verteilte Dateisysteme und blockchainähnliche Strukturen

Die Hardlink-Behandlung in [1] durch den Visited-Set-Mechanismus findet eine direkte Entsprechung im DNA-Pool: Ein Oligom, das als Subgraph mehrerer logischer Blöcke erkannt wurde, ist analog zu einem Hardlink — es wird im Pool nur einmal gespeichert, wird aber durch mehrere Signaturen adressiert.

Diese Struktur ähnelt einem *Content-Addressed Storage* (CAS) wie in Git, IPFS oder Ethereum-Merkle-Trees. Eine Erweiterung des DSS zu einem verteilten DNA-Speicher wäre konzeptionell naheliegend:

1. Jeder Pool-Knoten speichert einen Teil des Kerngenoms $\mathcal{M}$.
2. Die Subgraph-Signatur dient als inhaltliche Adresse (Content Hash).
3. Konsistenz über Knoten hinweg wird durch Signaturvergleich sichergestellt — exakt das Problem, das [1] für verteilte Dateisysteme löst.

Eine formale Theorie verteilter DNA-Subgraph-Speicher mit Konsistenzgarantien analog zum CAP-Theorem [9] ist eine offene Forschungsfrage.

12.14 Offene mathematische Probleme

Abschließend werden die wichtigsten offenen mathematischen Fragen benannt:

1. **Untere Schranke für erreichbare Komprimierung.** Wie groß ist der minimale Redundanzgrad $\rho_{\min}$, der für eine gegebene Verteilung $\mathcal{D}$ über Datenblöcke erreichbar ist? Dies hängt mit der Kolmogorov-Komplexität der Eingabe zusammen.

2. **NP-Härte des maximalen Kerngenoms.** Die Berechnung des optimalen maximalen Kerngenoms $\mathcal{M}$ unter $\preceq_{\text{rot}}$ ist möglicherweise NP-schwer (Reduktion auf Maximum-Weight-Clique). Formale Komplexitätsklassifikation, Inapproximierbarkeitsresultate und PTAS-Schemata stehen aus.
3. **Online-Subgraph-Komprimierung mit Approximationsgarantie.** Ein Online-Algorithmus, der Blöcke sequenziell verarbeitet und eine α -Approximation des optimalen Kerngenoms aufrecht erhält, fehlt bisher. Verbindungen zu Streaming-Algorithmen für Graphprobleme sind zu erwarten.
4. **Topologische DNA-Adressierung.** Können dreidimensionale DNA-Origami-Strukturen Datengraphen G_D entsprechen, deren Subgraph-Signatur geometrische Invarianten (Verschlingungszahl, Genus) kodiert? Dies könnte zu einer *topologischen DNA-Adressierung* führen — einem völlig neuen Adressierungsparadigma.
5. **Quantitative Theorie der GC-Ausgewogenheit.** Für welche Wahrscheinlichkeitsverteilungen $\mathcal{P}$ über $\mathcal{B}^{2k}$ gilt $\mathbb{E}_{D \sim \mathcal{P}}[|GC(\phi(D))|/k] = 1/2$? Eine vollständige Charakterisierung würde den Inversionsschritt im Schreib-Algorithmus durch eine optimale Bijektion ϕ^* ersetzen.
6. **Konvergenzgeschwindigkeit zur Shannon-Grenze.** Wie schnell konvergiert $R_{\text{DSS}}(k) \rightarrow C_{\text{DSS}}$ in der Oligolänge k ? Eine quantitative Schranke der Form $C_{\text{DSS}} - R_{\text{DSS}}(k) = O(k^{-\alpha})$ für ein explizites $\alpha > 0$ fehlt bisher.
7. **Online-Aufzug mit Approximationsgarantie (Abschnitt 12.1, Punkt A).** Existiert ein Online-Aufzugsalgorithmus für den DNA-Pool, der ohne Vorabkenntnis der Anfrageverteilung eine $(1 + \varepsilon)$ -Approximation der optimalen Sweep-Reihenfolge garantiert? Für den metrischen Raum $(\mathbb{N}^k, \leq_{\text{lex}})$ der Signatur-Ordnung sind die exakten Approximationsschranken noch offen. Bekannte Online-TSP-Resultate [11] liefern Faktor 2; ob für die Signatur-Ordnung eine bessere Konstante gilt, ist eine offene Frage.
8. **Konvergenz des adaptiven Ringpools (Abschnitt 12.2, Punkt A).** Konvergiert der adaptiv rebalancierende Ringpool nach T_{adapt} Operationen zur optimalen Segmentverteilung $m_k^* = M \cdot p_k$? Analogien zu Splay-Trees [13] legen amortisierte $O(\log n)$ -Rebalancierungskosten nahe — ein formaler Beweis steht aus.
9. **Byzantine Fault Tolerant Ringpool (Abschnitt 12.2, Punkt F).** Wie viele ausgefallene Knoten f kann ein verteilter Ringpool tolerieren, während er noch konsistente Antworten liefert? Für Consistent Hashing [14] ist $f < n/3$ bekannt; ob dieselbe Schranke für den Ringpool unter der zyklischen Gruppenstruktur $\mathbb{Z}_n$ gilt, ist offen.
10. **Rotationsstabilität für strukturierte Daten (Proposition 12.1).** Der Beweis von Proposition 12.1 setzt einen hinreichend großen LCS-Gap voraus. Für strukturierte Genome mit kleinem Gap fehlt eine scharfe untere Schranke für $\delta_{\text{LCS}}(k^*, k^* + 1)$ als Funktion der Graphstruktur von G_D .

13 Zusammenfassung

Diese Arbeit zeigt, dass die Ergebnisse der strukturellen Dateisystem-Deduplizierung [1] in der DNA-basierten Datenspeicherung [2] nicht nur Anwendung finden, sondern *das tragende mathematische Fundament* des gesamten Systems bilden.

Die zentralen Beiträge dieser Arbeit sind:

1. **Strukturelle Dualität** (Theorem 3.1): Inode-Graphen und DNA-Datengraphen sind durch eine surjektive, signaturerhaltende Abbildung Φ verbunden. Der Subgraph Algorithmus arbeitet in beiden Domänen auf exakt denselben algebraischen Objekten.
2. **DNA-Adressierung** (Lemma 4.1): Die aus [1] stammende Injektivität der Spalten-signatur garantiert die Eindeutigkeit aller DNA-Oligomer-Adressen.
3. **Komprimierungstheorem** (Satz 5.1): Die aus dem SDD-Algorithmus übernommene Subgraph-Inklusion $\preceq_{\text{rot}}$ reduziert den DNA-Pool auf den Faktor $(1 - \rho)^{-1}$ — vollständig bewiesen ohne Informationsverlust.
4. **Kanalkapazität** (Satz 6.1): Der Komprimierungsfaktor aus [1] erhöht die effektive Shannon-Kapazität des DNA-Kanals um den Faktor $(1 + \log_2(1 + \rho^{-1})/k)$.
5. **Haltbarkeitsmodell** (Satz 7.1): Der Redundanzgrad ρ aus [1] bestimmt direkt die garantierte Lebensdauer $t^* = -\ln(1 - \rho)/\lambda$ des DNA-Archivs.
6. **Gesamtoptimalität** (Satz 8.1): Das Gesamtsystem ist in allen vier Dimensionen (Kodierung, Komprimierung, Retrieval, Kapazität) asymptotisch optimal.
7. **Aufzug-Prinzip** (Satz 10.1, Satz 10.3): Eine Menge von N_{req} Blockanfragen kann durch signaturbasierte Sortierung und einen einzigen Sweep um den Faktor N_{req} gegenüber Einzelabruf beschleunigt werden. Das Aufzug-Prinzip ist unter dem Sweep-Kostenmodell beweisbar optimal.
8. **Zyklische Poolorganisation: Ringpool** (Satz 11.1, Theorem 11.1, Satz 11.2): Die zyklischen Rotationsmatrizen R_k des Subgraph Algorithmus dienen als physikalisches Adressierungsprinzip. Der Ringpool partitioniert den DNA-Pool in n gleichgroße Segmente. Lookup-Zeit für Blöcke mit bekannter Rotation: $O(\log(M/n))$ statt $O(\log M)$ — eine logarithmische Verbesserung um Faktor $\log n$. In Kombination mit dem Aufzug-Prinzip werden Zugriffszeiten von unter 6 Sekunden für Pools mit 10^{12} Oligomeren und 10^6 gleichzeitigen Anfragen erreicht (Satz 11.3).

Die übergeordnete Erkenntnis ist dabei: Der Subgraph Algorithmus aus [1] ist in [2] nicht bloß ein wiederverwendetes Werkzeug, sondern ein *mathematischer Kern*, dessen Eigenschaften — Injektivität, Rotationsinvarianz, $O(n^3)$ -Optimalität — direkt und vollständig in die DNA-Domäne übertragen werden. Die Arbeit [1] verhält sich zu [2] wie ein Theorem zu seinem direkten Korollar.

Literatur

- [1] Stephan Epp, *Strukturelle Deduplizierung von Dateisystem-Graphen mittels zyklischer Subgraph-Erkennung*, April 2026.

- [2] Stephan Epp, *DNA-basierte Datenspeicherung mittels des Subgraph Algorithmus*, April 2026.
- [3] A. Abboud, A. Backurs, V. V. Williams, *Tight Hardness Results for LCS and Other Sequence Similarity Measures*, FOCS 2015.
- [4] G. M. Church, Y. Gao, S. Kosuri, *Next-Generation Digital Information Storage in DNA*, Science, 337(6102):1628, 2012.
- [5] N. Goldman et al., *Towards practical, high-capacity, low-maintenance information storage in synthesized DNA*, Nature, 494:77–80, 2013.
- [6] L. Organick et al., *Random access in large-scale DNA data storage*, Nature Biotechnology, 36:242–248, 2018.
- [7] D. Deamer, M. Akeson, D. Branton, *Three decades of nanopore sequencing*, Nature Biotechnology, 34:518–524, 2016.
- [8] I. S. Reed, G. Solomon, *Polynomial Codes Over Certain Finite Fields*, J. Soc. Ind. Appl. Math., 8(2):300–304, 1960.
- [9] E. A. Brewer, *Towards robust distributed systems*, PODC 2000 Keynote.
- [10] A. S. Tanenbaum, H. Bos, *Modern Operating Systems*, 4th ed., Pearson, 2014.
- [11] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed., Addison-Wesley, 1998.
- [12] L. K. Grover, *A fast quantum mechanical algorithm for database search*, STOC 1996, pp. 212–219.
- [13] D. D. Sleator, R. E. Tarjan, *Self-adjusting binary search trees*, J. ACM, 32(3):652–686, 1985.
- [14] D. Karger et al., *Consistent hashing and random trees*, STOC 1997, pp. 654–663.
- [15] S. M. H. T. Yazdi, Y. Yuan, J. Ma, H. Zhao, O. Milenkovic, *A Rewritable, Random-Access DNA-Based Storage System*, Scientific Reports, 5:14138, 2015.
- [16] Y. Erlich, D. Zielinski, *DNA Fountain enables a robust and efficient storage architecture*, Science, 355(6328):950–954, 2017.
- [17] M. Blawat et al., *Forward Error Correction for DNA Data Storage*, Procedia Computer Science, 80:1011–1022, 2016.
- [18] J. Bornholt et al., *A DNA-Based Archival Storage System*, ASPLOS 2016, pp. 637–649.
- [19] R. N. Grass, R. Heckel, M. Puddu, D. Paunescu, W. J. Stark, *Robust Chemical Preservation of Digital Information on DNA in Silica with Error-Correcting Codes*, Angew. Chemie Int. Ed., 54(8):2552–2555, 2015.
- [20] L. Ceze, J. Nivala, K. Strauss, *Molecular digital data storage using DNA*, Nature Reviews Genetics, 20:456–466, 2019.

- [21] O. Milenkovic, S. Levy, J. Bruck, *Coding for DNA Storage*, IEEE Trans. Inf. Theory, 66(11):6705–6710, 2020.
- [22] K. A. S. Immink, O. Milenkovic, *Codes for DNA Storage Channels*, IEEE Trans. Inf. Theory, 66(11):6915–6936, 2020.
- [23] D. Boneh, C. Dunworth, R. J. Lipton, *Breaking DES using a molecular computer*, DNA Based Computers, DIMACS 1996, pp. 37–65.
- [24] L. M. Adleman, *Molecular computation of solutions to combinatorial problems*, Science, 266(5187):1021–1024, 1994.
- [25] P. W. K. Rothmund, *Folding DNA to create nanoscale shapes and patterns*, Nature, 440:297–302, 2006.
- [26] K. N. Lin et al., *Dynamic and scalable DNA-based information storage*, Nature Communications, 11:2981, 2020.
- [27] L. Anavy et al., *Data storage in DNA with fewer synthesis cycles using composite DNA letters*, Nature Biotechnology, 37:1229–1236, 2019.
- [28] K. Srinivasan, J. Bruck, P. W. K. Rothmund, *Algorithms for reading and writing compressed representations of DNA data*, ISIT 2021, pp. 3048–3053.
- [29] B. Wang et al., *High capacity DNA data storage with variable-length Oligonucleotides using a Plug-and-Play encryption strategy*, Small, 15(29):1901654, 2019.